PSG COLLEGE OF TECHNOLOGY, COIMBATORE

Department of Information Technology

23I006 – Deep Learning: Mini Project (Conference Paper)

Privacy-Preserving Human Activity Recognition from mmWave Radar Point Clouds using Lightweight Point Transformers

Technical Development and Decision Report

Submitted by (Team 10)

Abarajitha S K	23I301
G Aswath Rupan	23I321
Indu Vadana M D	23I323
Murugan S	23I338

B.Tech Information Technology

November 2026

Abstract

This report documents the complete development of a privacy-preserving human activity recognition system based on mmWave radar point clouds, carried out for the 23I006 Deep Learning mini project. It records every important decision, the alternatives considered, the failures met and how each was diagnosed, corrected and verified. Radar returns are pre-processed with an antenna-return filter, Doppler-based person centring and spatial gating, which remove about half of all returns as static clutter. The proposed STPT-Lite applies self-attention within short temporal segments and across segment tokens and adds an explicit motion descriptor. It is distilled from a larger teacher, trained with noise-robust augmentation and stored with INT8 weights (117.5 K parameters, 36.6 MFLOPs, 2.58 ms per window on one CPU thread). Under leakage-free protocols with three seeds, it reaches 96.83% on RadHAR and 49.56% top-1 on 49 MiliPoint actions of unseen people, against 32.57% for the best re-implemented baseline. An automated audit shows that random window splits place most test frames inside training windows, which explains the large spread of published results. The report also covers the robustness study, ablations, cross-dataset transfer, explainability, the working demonstration, testing and the remaining limitations.

Acknowledgement

We thank the faculty of the Department of Information Technology, PSG College of Technology, for their guidance in the 23I006 Deep Learning course. We also thank the authors of the RadHAR (UCLA NESL, BSD-3-Clause licence) and MiliPoint (MIT licence) datasets for making their data and code public.

Contents

Abstract i

Acknowledgement ii

Abbreviations x

1 Project Overview 1

1.1 Background . . . 1
1.2 Problem statement . . . 1
1.3 Motivation . . . 1
1.4 Objectives . . . 2
1.5 Scope . . . 2
1.6 Expected outcomes . . . 2
1.7 System requirements . . . 2

2 Literature Review and Technology Investigation 3

2.1 Radar-based HAR . . . 3
2.2 Point-cloud architectures . . . 4
2.3 Noise-handling approaches . . . 4
2.4 Lightweight models . . . 4
2.5 Research gap . . . 4
2.6 Technology alternatives . . . 5

3 Dataset Selection and Analysis 6

3.1 Why these datasets . . . 6
3.2 Availability and licences . . . 6
3.3 Data format and acquisition . . . 6
3.4 Measured characteristics . . . 7
3.5 Class distribution . . . 8
3.6 Noise characteristics (measured, not assumed) . . . 8
3.7 Can the datasets be combined? . . . 9

3.8 Existing evaluation protocols 9
3.9 Dataset limitations 9

4 System Architecture and Design Decisions 10
4.1 Overall architecture 10
4.2 Components 10
4.3 Data-processing architecture 12
4.4 Training and inference architecture 12
4.5 Application architecture 12
4.6 Edge deployment 12
4.7 Decision traceability matrix 13

5 Data Pre-processing and Noise Handling 15
5.1 Windowing 15
5.2 Region-of-interest (ROI) filter 15
5.3 Person centring with moving returns 16
5.4 Spatial gating (clutter removal) 16
5.5 Temporal segmentation and point sampling 16
5.6 Feature normalisation 16
5.7 Training-time augmentation 17
5.8 Controlled noise conditions for evaluation 17
5.9 Methods investigated but not used 17

6 Model Selection and Architecture 18
6.1 Candidate architectures 18
6.2 Selection rationale 18
6.3 Layer-by-layer architecture of STPT-Lite 19
6.4 Computational complexity 19
6.5 Training variants and final model selection 20
6.6 Variants 20

7 Training and Optimisation 21
7.1 Environment 21
7.2 Hyper-parameters and their justification 21

7.3 Overfitting prevention . . . 22
7.4 Training-stability investigation . . . 22

8 Data Leakage Prevention 24
8.1 Splitting methodology . . . 24
8.2 Restrictions . . . 24
8.3 Automated audit . . . 24
8.4 Findings . . . 25

9 Implementation Details 27
9.1 Repository structure . . . 27
9.2 Module descriptions . . . 27
9.3 Key code . . . 29
9.4 Reproducibility . . . 29

10 Failed Experiments, Errors and Rectification History 30
10.1 Most consequential issues . . . 38

11 Experimental Results 40
11.1 Main comparison . . . 40
11.2 Per-class results and confusion . . . 41
11.3 Training and validation behaviour . . . 42
11.4 Ablation, protocol study and transfer . . . 42
11.5 Noise robustness . . . 43
11.6 Explainability . . . 45
11.7 Computational efficiency . . . 46

12 Working Demonstration 49
12.1 Purpose and integrity . . . 49
12.2 How to run . . . 49
12.3 Application structure . . . 49
12.4 Input validation and error handling . . . 50
12.5 Bundled samples . . . 50
12.6 Screenshots . . . 50

13 Testing and Quality Assurance **55**

13.1 Test strategy 55

13.2 Results 56

13.3 Quality-assurance matrix 56

14 Limitations and Future Enhancements **58**

14.1 Limitations 58

14.2 Future enhancements 58

15 Conclusion **60**

References **63**

A Configuration, Installation and Commands **64**

A.1 Default configuration (`configs/default.yaml`) 64

A.2 Installation 64

A.3 Data preparation 64

A.4 Training and evaluation 65

A.5 Inference and demonstration 65

B Test Log **66**

List of Figures

3.1 RadHAR. (a) points per frame by class; (b) non-zero Doppler values, which take only a few quantised values (± 0.6, ± 1.2 m/s); (c) density of all returns in the range–height plane (log scale): the person sits at 1–2 m, and static clutter bands lie at about 2.5 m and 4.5 m. 7

3.2 MiliPoint. (a) points per frame (median 6); (b) labelled time per action; (c) labelled minutes per subject and the subject-disjoint split (blue train, orange validation, red test). 8

3.3 One pre-processed 2-s RadHAR test window per class (points coloured by time). 8

4.1 End-to-end system architecture. 10

5.1 One raw 2-s RadHAR window (left), with antenna self-returns (red), static clutter removed by gating (grey), person returns (blue) and moving returns (black circles), and the resulting model input (right). 15

11.1 Row-normalised confusion matrices of the proposed STPT-Lite+KD+robust (seed 0). 41

11.2 Validation macro-F1 per epoch (seed 0) for every model. RadHAR saturates within a few epochs; on MiliPoint the STPT models converge between epochs 15 and 30, while the PointNet family keeps a large gap. 42

11.3 Accuracy under controlled perturbations (identical perturbed test samples for every model). Top: MiliPoint; bottom: RadHAR. 47

11.4 Temporal attention vs. occlusion importance, per-point gradient saliency, and t-SNE of embeddings (RadHAR test). 48

11.5 Accuracy vs. parameters and latency. 48

12.1 Demonstration (real screenshot), Tab 1: raw window with antenna returns, gated clutter and kept person returns; pre-processed model input. 51

12.2 Demonstration (real screenshot), Tab 2: live recording-level prediction, latency and per-window class probabilities. 52

12.3 Demonstration (real screenshot), Tab 3: controlled noise injection; clean vs. noisy point cloud and predictions. 53

12.4 Demonstration (real screenshot), Tab 4: model card with held-out test metrics and confusion matrix. 54

List of Tables

2.1 Reviewed mmWave point-cloud HAR literature (accuracies as reported by the authors under their own protocols). 3

2.2 Technology choices and alternatives considered. 5

3.1 Dataset availability. 6

3.2 Dataset characteristics measured by `scripts/eda.py`. 7

4.1 Component design decisions. 10

4.2 Decision traceability matrix. 13

5.1 Evaluation perturbations (applied to the held-out test set only). 17

6.1 STPT-Lite layers (batch B, $G = 12$, $P = 32$, $d = 64$, $H = 4$ heads, K classes). Parameter counts for $K = 5$. 19

6.2 Model variants used in the experiments (parameters for 5 / 49 classes). 20

7.1 Computational environment. 21

7.2 Training configuration. 21

8.1 Automated leakage-audit results. 25

9.1 Modules: purpose, interfaces, error handling and verification. 27

10.1 Issue-resolution log. 30

11.1 RadHAR official test recordings (3,565 windows, 5 classes). Mean $\pm$ std over 3 seeds. Sig.: number of seeds in which the Holm-corrected McNemar test against the proposed model gives $p < 0.05$; $\uparrow$ ours better / $\downarrow$ ours worse. Latency: median, batch 1, one CPU thread. 40

11.2 MiliPoint, subject-disjoint test (subjects 8–10, 3,472 windows, 49 actions). Same conventions as Table 11.1. 40

11.3 Per-class test precision, recall and F1 (%) of the proposed model (STPT-Lite+KD+robust) on RadHAR (mean over seeds). 41

11.4 Ablation of STPT-Lite (no distillation): test accuracy (%), mean $\pm$ std over seeds (variants: 2 on RadHAR, 2 on MiliPoint; full model: 3), change w.r.t. the full model trained with the same seeds in brackets; † single run. 43

11.5 MiliPoint (49 actions) top-1 accuracy (%) of the same models under three splits. Published numbers: 34.45 (PointNet++, [1]) and 98.25 (MAK-GCN, random 80/10/10, [2]). 43

11.6 Cross-dataset transfer (STPT-Lite, *xyz* input): test accuracy (%) when fine-tuning on a fraction of target training recordings/runs, from scratch vs. initialised from the MiliPoint model (mean ± std over 2 seeds; n=1 marks a single run). Scratch at 100% is the *xyz*-only ablation run. 44

11.7 Robustness on MiliPoint (unseen subjects): test accuracy (%) / prediction stability (% of predictions unchanged w.r.t. clean), mean over seeds. 44

11.8 Robustness on RadHAR: test accuracy (%) / prediction stability (% of predictions unchanged w.r.t. clean), mean over seeds. 45

13.1 Test levels. 55

13.2 Test results (generated by `scripts/make_qa.py`). 56

13.3 Final quality-assurance matrix (evidence-based). 56

Abbreviations

CI	Confidence interval
CLS	Classification token
CPU / GPU	Central / graphics processing unit
DGCNN	Dynamic graph CNN
FLOP	Floating-point operation
FMCW	Frequency-modulated continuous wave
FPS	Farthest-point sampling (or frames per second, from context)
HAR	Human activity recognition
INT8 / FP32	8-bit integer / 32-bit floating point
KD	Knowledge distillation
k-NN	k nearest neighbours
MLP	Multi-layer perceptron
mmWave	Millimetre wave (30–300 GHz; here 60–81 GHz radar)
NLL	Negative log-likelihood
ROI	Region of interest
STPT	Spatio-Temporal Point Transformer (proposed)
t-SNE	t-distributed stochastic neighbour embedding

Chapter 1

Project Overview

1.1 Background

Human activity recognition (HAR) at home supports elderly-care monitoring, rehabilitation and smart-home automation. Camera-based HAR is accurate, but it records identifiable images of people in private spaces, it fails in darkness, and many users do not accept it. Wearable sensors avoid images but must be worn and charged. Single-chip frequency-modulated continuous-wave (FMCW) mmWave radars operating at 60–81 GHz (e.g. the Texas Instruments IWR1443/IWR1843 used by both datasets in this project) sense through darkness and light clothing. For each frame they output a small *point cloud*: a set of detections with a 3-D position, a radial (Doppler) velocity and an intensity. The output contains no image and no visual identity, so the sensing is privacy-preserving by construction.

1.2 Problem statement

Given a short sequence of radar point-cloud frames, the task is to classify the human activity being performed, accurately and at a computational cost that suits an edge device. The difficulty has four sources:

- **Sparsity**: a frame holds only about 6 (MiliPoint) to 21 (RadHAR) returns (measured in Chapter 3).
- **Noise**: many returns are antenna self-reflections, static clutter or multipath rather than the person. Our analysis shows that about half of all RadHAR returns are not on the person (Chapter 5).
- **Temporal nature**: the activity shows up in how the sparse cloud moves over time, not in the shape of a single frame.
- **Evaluation leakage**: samples are overlapping windows cut from long recordings, so careless splitting puts near-identical samples in both the training and the test data (Chapter 8).

1.3 Motivation

Published results on the same public radar datasets vary enormously. On MiliPoint's 49-action task, the dataset authors report at most 34.45% top-1 [1], while a 2025 graph-network paper reports 98.25% under a random 80/10/10 split [2]. A model that is accurate only because of

leakage is useless in a real home. This project therefore aims to show what a lightweight model achieves under *honest* evaluation and how robust it is to realistic radar disturbances.

1.4 Objectives

1. Build a reproducible pipeline for two public mmWave point-cloud datasets (MiliPoint as the primary dataset, RadHAR as the secondary one).
2. Design radar-specific pre-processing that removes non-human returns without removing human motion.
3. Develop a lightweight point-transformer classifier suitable for edge deployment and compare it with established baselines (PointNet, PointNet++, Point Transformer, DGCNN, PointNet+BiLSTM and the RadHAR voxel CNN-LSTM).
4. Evaluate under leakage-free protocols with three seeds, paired significance tests, ablations and a dedicated noise-robustness study.
5. Deliver a saved model, an inference script and a working interactive demonstration, together with a research paper, this report and a presentation.

1.5 Scope

In scope: activity classification of single-person radar point-cloud sequences using the label sets of RadHAR (5 activities) and MiliPoint (49 actions); CPU-based efficiency measurement; robustness to synthetic perturbations of recorded data. Out of scope: fall detection (neither dataset has fall labels), multi-person scenes, on-device hardware deployment, and clinical validation. No claim of clinical suitability is made.

1.6 Expected outcomes

A trained and saved model with measured accuracy on held-out data; quantified robustness under controlled noise; an automated leakage audit; a demonstration application that runs the real model; and complete documentation.

1.7 System requirements

- Python $\geq$ 3.10, PyTorch $\geq$ 2.1 (developed with 2.15), NumPy, SciPy, scikit-learn, pandas, Matplotlib, PyYAML; Streamlit and Plotly for the demo.
- No GPU required. All experiments in this report ran on a 2-core CPU with 7 GB RAM (Section 7.1).
- Disk: about 1 GB for the RadHAR raw data and 0.4 GB for MiliPoint.

Chapter 2

Literature Review and Technology Investigation

2.1 Radar-based HAR

Radar HAR has historically used micro-Doppler spectrograms or range-Doppler maps; Ullmann et al. [3] survey continuous radar HAR. Low-cost single-chip radars output point clouds directly, which made point-cloud HAR popular. RadHAR [4] recorded five activities with a TI IWR1443 and compared an SVM, an MLP, a BiLSTM and a time-distributed CNN+BiLSTM on $10 \times 32 \times 32$ voxel grids; the CNN+BiLSTM performed best (90.47%). Table 2.1 lists the later work we reviewed.

Table 2.1: Reviewed mmWave point-cloud HAR literature (accuracies as reported by the authors under their own protocols).

Work	Method	Reported	Note
RadHAR 2019 [4]	Voxels + TD-CNN + BiLSTM	90.47% (RadHAR)	dataset paper; window-level validation
MMPoint-GNN 2021 [5]	dynamic-edge GNN per frame	–	graph on sparse frames
Lee & Kim 2022 [6]	GNN with pre-trained 3-D joints	96.55% (RadHAR)	needs joint supervision
Tesla-Rapture 2023 [7]	light temporal graph model	gestures	sparse radar point clouds
Pantomime 2021 [8]	point-cloud gesture recogniser	gestures	sparse mmWave clouds
Gu et al. 2024 [9]	Lite-PointNet + BiLiLSTM	95.12% (RadHAR)	79.7 K params
Lim et al. 2024 [10]	pillars + DS-CNN, 4-bit FPGA	94.79% (own data)	22.3 K params
MAK-GCN 2025 [2]	multi-head adaptive graph conv.	97.54% RadHAR; 98.25% MiliPoint	random 80/10/10 on MiliPoint
OG-PCL 2025 [11]	occupancy-gated convolution	91.75% (RadHAR)	5-fold CV; 0.83 M params
Star graph 2025 [12]	star graph around virtual centre	94.27% (own data)	subject-wise split
MiliPoint 2023 [1]	benchmark (DGCNN, PointNet++, . . .)	34.45% (49 actions)	dataset paper

2.2 Point-cloud architectures

PointNet [13] applies a shared MLP to every point and max-pools the results, which gives a permutation-invariant set function. It is cheap, but it has no notion of local neighbourhoods. **PointNet++** [14] adds hierarchical set abstraction: farthest-point sampling (FPS) picks centroids, a ball query groups the neighbours of each centroid, and a mini-PointNet runs on every group. **DGCNN** [15] builds a k-NN graph dynamically in feature space and applies edge convolutions. **Point Transformer** [16] replaces convolution with vector self-attention over k-NN neighbourhoods, using relative positional encodings. PCT [17], PTv2/PTv3 [18, 19], PointNeXt [20] and PointMLP [21] extend these ideas. P4Transformer [22] handles point-cloud *videos* with spatio-temporal tubes. All of these assume dense clouds (hundreds to thousands of points) and depend on FPS and k-NN, whose cost grows quadratically or worse with the number of points and which map poorly to micro-controllers.

2.3 Noise-handling approaches

Typical radar point-cloud pipelines use static-clutter removal (often inside the radar firmware), range or angle region-of-interest filters, statistical or radius outlier removal, DBSCAN clustering to isolate a person, and multi-frame accumulation to counter sparsity [4, 7]. On the learning side, point dropout, jitter and random resampling are standard augmentations for point-cloud networks [13, 14].

2.4 Lightweight models

Knowledge distillation [23] trains a small student to match a large teacher's softened outputs. Quantisation [24] reduces the weights (and optionally the activations) to 8 bits. Lim et al. [10] show that 4-bit radar HAR runs on FPGAs.

2.5 Research gap

1. Radar-specific structure (extreme sparsity, a direct Doppler measurement, static clutter) is rarely exploited explicitly. Most models reuse dense-point backbones or voxel grids.
2. Protocols are inconsistent and frequently leaky (window-level random splits), so published numbers are not comparable.
3. Robustness to radar disturbances, cross-dataset transfer and edge cost are rarely reported together.

2.6 Technology alternatives

Table 2.2: Technology choices and alternatives considered.

Need	Chosen	Alternatives	Reason
DL framework	PyTorch 2.15	TensorFlow/Keras (used by RadHAR)	dynamic graphs, simple custom attention; course familiarity
Point ops	pure PyTorch (own FPS, ball query, k-NN)	PyTorch Geometric, torch-cluster	PyG/torch-cluster need compiled CUDA/CPU wheels; one dependency-free implementation runs everywhere
Demo	Streamlit + Plotly	Gradio, Flask + JS	fastest route to an interactive 3-D app; built-in test harness (AppTest)
Paper/report	LaTeX (IEEEtran / report)	Word	exact IEEE template, reproducible tables generated from results
Statistics	SciPy exact binomial (McNemar), bootstrap	t-tests on seeds	paired test on the same test windows is more powerful and makes fewer assumptions

Chapter 3

Dataset Selection and Analysis

3.1 Why these datasets

The project brief prescribes MiliPoint (primary) and RadHAR (secondary). Both are public, both are produced by Texas Instruments single-chip radars, and both provide point clouds rather than spectrograms. Together they cover two complementary regimes. MiliPoint is a large benchmark (49 actions, 11 subjects) that records *who* performed each sequence, which makes a subject-disjoint split possible. RadHAR is small (5 activities) but provides Doppler and intensity for every point and an official test partition used by most prior work.

Alternatives considered. MM-Fi [25] (multi-modal, 27 actions) and mmBody [26] (mesh reconstruction) were reviewed. MM-Fi is large (tens of GB), is aimed at multi-modal fusion and needs a separate access request. mmBody has no activity labels. Neither was required by the brief, so neither was used. No dataset was replaced.

3.2 Availability and licences

Table 3.1: Dataset availability.

Dataset	Source	Licence	Access in this project
RadHAR	github.com/nesl/RadHAR (raw data in repository)	BSD-3-Clause	`git clone`, 975 MB
MiliPoint	github.com/yizzfz/MiliPoint (code, `id.json`); data zip on Google Drive	MIT	Drive blocked by the network proxy; zip downloaded manually by the team and staged (Issue #3)

3.3 Data format and acquisition

RadHAR. Raw files are text dumps of ROS `PointCloud` messages, with one message per detected point (`seq`, timestamp, `point_id`, x, y, z, range, velocity, Doppler bin, bearing, intensity). A new radar frame starts whenever `point_id` resets to 0. The radar is a TI IWR1443 using the `ti_mmwave_rospkg` driver. The dataset paper reports two subjects, but the files carry no subject identifier.

MiliPoint. Nineteen pickle files, each a list of frames with `x` (points $[n, 3]$) and `y` (18 pose keypoints), plus a global `action_label.npy` (one label per frame; -1 = unlabelled) and `id.json` mapping the 11 subjects to files. The radar is a TI IWR1843 at 24 frames per second; subjects followed an instructional video of 49 actions of 30 s each [1]. Action *names* are not distributed with the data, so actions are referred to by their numeric ID (0–48).

3.4 Measured characteristics

Table 3.2: Dataset characteristics measured by `scripts/eda.py`.

	RadHAR	MiliPoint
Subjects	2 (paper); IDs not available	11 (IDs available)
Classes	5: boxing, jumping jacks, jump, squats, walk	49 actions (IDs 0–48)
Recordings	200 (142 train, 58 test)	19 session files; 1,139 single-action runs
Duration / frames	71.5 min train + 21.4 min test	545,059 frames; 212,920 labelled
Frame rate	30 Hz nominal (irregular timestamps)	24 Hz
Points per frame (median / mean)	21 / 21.4	6 / 6.5
Point features	x, y, z, Doppler (quantised), intensity	x, y, z only
Coordinate frame	x forward, y lateral, z up	x lateral, y forward, z up (verified from keypoints)
Missing / corrupted	none found; 34.6% antenna self-returns	file 4 fully unlabelled; files 1, 6 partly

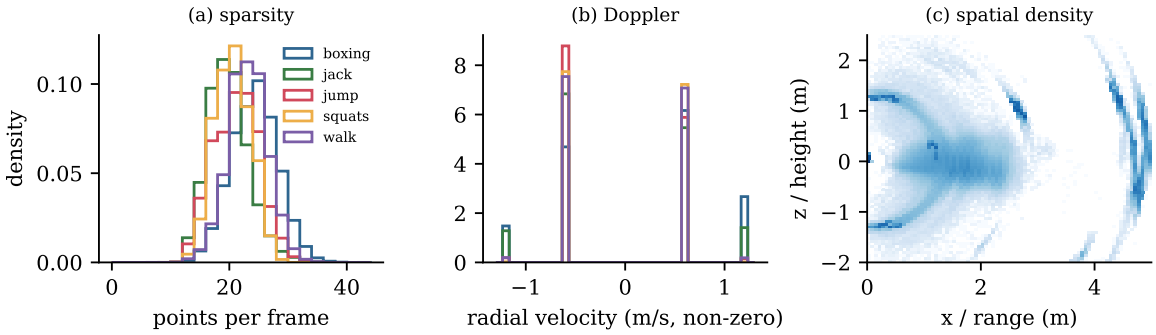


Figure 3.1: RadHAR. (a) points per frame by class; (b) non-zero Doppler values, which take only a few quantised values (± 0.6, ± 1.2 m/s); (c) density of all returns in the range–height plane (log scale): the person sits at 1–2 m, and static clutter bands lie at about 2.5 m and 4.5 m.

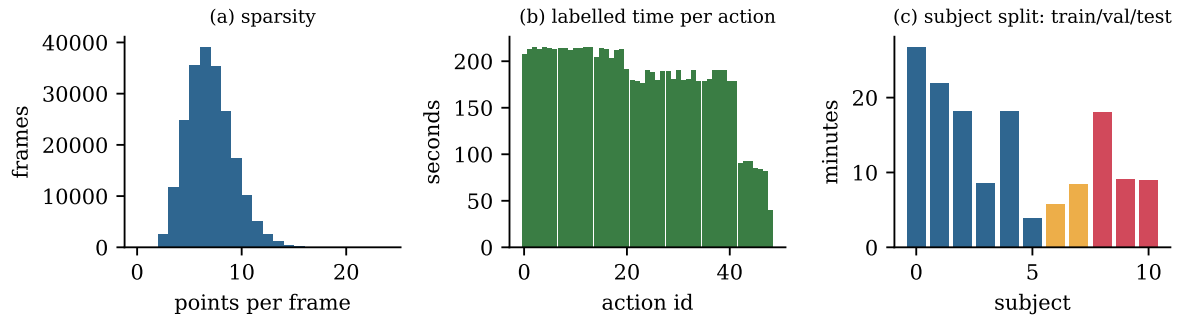


Figure 3.2: MiliPoint. (a) points per frame (median 6); (b) labelled time per action; (c) labelled minutes per subject and the subject-disjoint split (blue train, orange validation, red test).

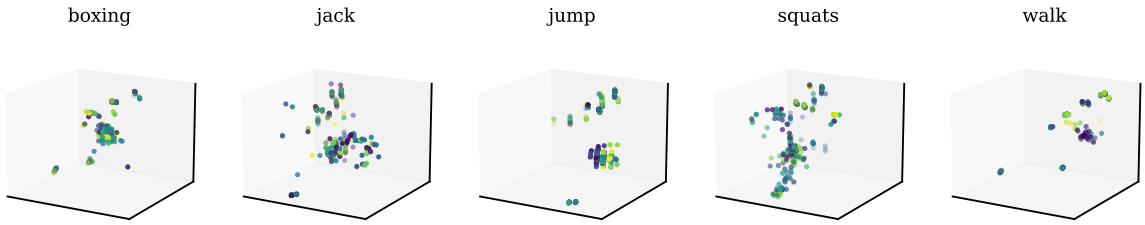


Figure 3.3: One pre-processed 2-s RadHAR test window per class (points coloured by time).

3.5 Class distribution

RadHAR training recordings per class (boxing, jack, jump, squats, walk) are 26/28/26/29/33 and test recordings 13/10/11/10/14. The test windows per class are 714/673/734/674/770, which is close to balanced. Some training recordings are much longer than the standard 20 s (for example `boxing_184s.txt` and `___squats_186s.txt`). This turned out to matter for validation (Issue #16–#17). MiliPoint actions 0–40 have about 200 s of labelled data each, and actions 41–48 fewer (Fig. 3.2b).

3.6 Noise characteristics (measured, not assumed)

- **Antenna self-returns**: 34.6% of RadHAR returns lie at $x < 0.3$ m.
- **Static clutter / multipath**: after the ROI filter, 50.1% of returns lie more than 1.2 m (horizontally) from the moving-point centre of the person. They form persistent clusters at fixed ranges with zero Doppler (Fig. 5.1).
- **Sparsity and variable density**: 15–28 returns per frame (5th–95th percentile) on RadHAR, and a median of 6 on MiliPoint.
- **Doppler quantisation**: velocities take a handful of discrete values.
- **Not observed**: corrupted files, NaN values or duplicated recordings (verified by the leakage audit, Chapter 8).

Random coordinate displacement, point dropout, false detections and frame loss are *not* directly measurable without ground-truth body surfaces. They are therefore studied as *controlled synthetic perturbations* of recorded data (Chapter 5), and are clearly distinguished from the measured noise above.

3.7 Can the datasets be combined?

Not as one training set. The label sets differ (5 vs 49 classes, and MiliPoint action names are not published, so no reliable label mapping exists). The radars differ (IWR1443 vs IWR1843), MiliPoint lacks Doppler and intensity, and the axes differ. We therefore run **independent experiments** per dataset and use the other dataset only for **representation transfer** after harmonising the axes and restricting the input to *xyz* (Chapter 11).

3.8 Existing evaluation protocols

RadHAR: an official train/test partition by recording, with validation on a random 20% of training *windows* (which overlap by 50/60 frames). MiliPoint: the published code concatenates labelled frames from all files and cuts them chronologically 80/10/10, so the test data contain subjects that also appear in training; other work uses random 80/10/10 splits [2]. Chapter 8 measures the leakage of each protocol.

3.9 Dataset limitations

RadHAR has no subject IDs and only two subjects. Its official test recordings were made on the same days as the training recordings, often minutes apart (Chapter 8). MiliPoint provides *xyz* only, and its action names are unknown. Neither dataset contains falls or multi-person scenes.

Chapter 4

System Architecture and Design Decisions

4.1 Overall architecture

Figure 4.1 shows the complete system. Offline, raw dataset files are parsed into per-recording frame arrays and cut into leakage-free splits. A data module turns every 2-s window into a fixed tensor. The training script fits a model and selects a checkpoint on validation data only. The analysis scripts evaluate the held-out test set, the noise conditions and the statistics. The exported model *bundle* (weights + pre-processing settings + class names) is the only artefact that the inference script and the demonstration application need.

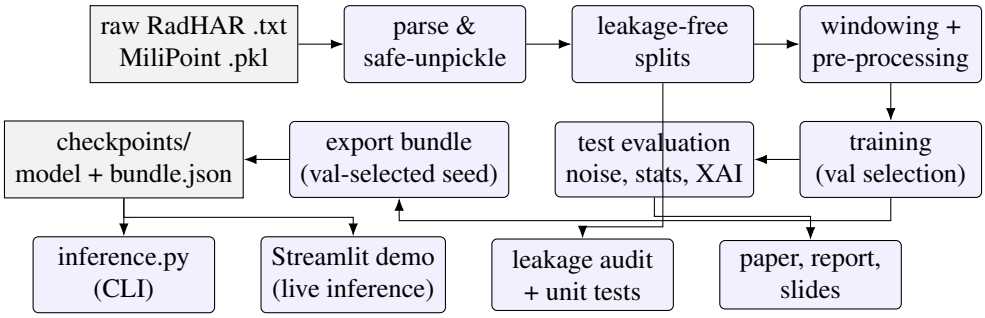


Figure 4.1: End-to-end system architecture.

4.2 Components

For each component, Table 4.1 gives what it is, why it is used, the alternatives considered, how it connects to the rest of the system, and its limitations.

Table 4.1: Component design decisions.

Component	What / why	Alternatives	Connections	Limitations
Parser (`radhar_parse.py`)	regex over ROS text; frame = run of `point_id` from 0. Needed because only raw dumps exist	the original voxel script (slow, keeps no Doppler)	produces `radhar_frames.npz` for the data module and the demo	relies on the ROS dump format

Component	What / why	Alternatives	Connections	Limitations
Safe unpickler	loads MiliPoint pickles but allows only numpy reconstruction	plain `pickle.load` (allows code execution)	feeds `data_milipoint.py`	a whitelist must be extended if the file format changes
Data module (`data_radhar.py`)	windowing, ROI, person-centring, gating, per-segment sampling, augmentation	voxel grids; fixed zero-padding	used by training, evaluation, inference and the demo, so pre-processing is identical everywhere	2-s windows only
Models (`models.py`)	7 architectures behind one interface `f(x, mask)`	PyG implementations	built by `build(name)`; used by training and inference	pure-PyTorch FPS/k-NN are slower than CUDA kernels
Training (`train.py`)	AdamW, one-cycle, label smoothing, early stopping on val macro-F1, optional KD / robust augmentation	SGD, fixed epochs	reads `configs/default.yaml`; writes run directories	CPU-bound
Noise (`noise.py`)	seeded perturbations for evaluation and training	ad-hoc numpy noise	robustness study, robust training, demo	synthetic, not physical radar simulation
Inference (`inference.py`)	loads a bundle and applies the training pre-processing to raw input	re-implementing pre-processing in the app	CLI and demo	needs at least 2 s of data
Demo (`demo/app.py`)	interactive upload, visualisation, live prediction, noise injection	Gradio	imports `inference.py`	desktop browser; no live radar stream

Component	What / why	Alternatives	Connections	Limitations
Audit (`leakage_audit.py`)	automated overlap/duplicate/statistics checks	manual inspection	reads splits and run logs	cannot detect subject overlap without subject IDs

4.3 Data-processing architecture

Raw data → per-recording arrays (points, frame index, timestamps) → recording/subject-level split → window list (recording, start frame) → on-the-fly tensor construction. Building tensors on the fly keeps memory small and lets each epoch draw different point samples, which acts as augmentation.

4.4 Training and inference architecture

Training uses one process per configuration (one CPU thread) and a small job queue (`scripts/runner.py`) that runs two jobs in parallel, skips finished jobs and resolves dependencies (a distilled student waits for its teacher). Inference runs one forward pass per 2-s window (sliding by 1/3 s) and reports a recording-level majority vote.

4.5 Application architecture

The Streamlit application caches the loaded bundle, validates the uploaded file, calls `Bundle.windows_fro` (the same code path as evaluation) and renders Plotly 3-D views. It is described in Chapter 12.

4.6 Edge deployment

The proposed model has no data-dependent sampling (no FPS or k-NN), only dense matrix products on tensors of fixed size (12×32 points). This makes it straightforward to export to ONNX or TFLite-Micro-style runtimes. Its INT8 weights occupy 0.14 MB and one inference costs 36.6 MFLOPs; the measured single-thread CPU latency is 2.58 ms. Deployment on an actual micro-controller was out of scope and has not been verified.

4.7 Decision traceability matrix

Table 4.2 links every important decision to the alternatives that were investigated and to the evidence behind the final choice. "Issue #" refers to the development log (Chapter 10).

Table 4.2: Decision traceability matrix.

Decision	Alternatives investigated	Final selection	Reason	Supporting evidence
Dataset	MM-Fi, mmBody	MiliPoint (primary), RadHAR (secondary)	prescribed; public; subject IDs (MiliPoint); Doppler (RadHAR)	Ch. 3; licences BSD-3 / MIT
Combining datasets	joint training	separate experiments + *xyz* transfer	label sets, sensors and features differ	Ch. 3 analysis; transfer results (Table 11.4)
Input representation	voxel grids; single frames	12 segments × 32 points, 2-s window	frames too sparse (6–21 pts); voxels mostly empty	EDA; voxel baseline results
Noise handling	DBSCAN; statistical/radius outlier removal; plain centroid	ROI + moving-point centring + 1.2 m gating	clutter is static and far from the moving person	range histograms (Issue #13); Fig. 5.1; gating ablation
Model architecture	PointNet, PointNet++, DGCNN, Point Transformer, voxel CNN-LSTM, PointNet-LSTM	STPT-Lite (+KD)	accuracy/efficiency trade-off under identical protocol	Tables 11.1–11.2
Compression	dynamic INT8; no compression	KD + weight-only INT8	dynamic INT8 lost 7 pts and was slower	Issue #11; INT8 accuracy in results

Decision	Alternatives	Final	Reason	Evidence
Data splitting	random windows (RadHAR original, some MiliPoint work); chronological	recording-disjoint (RadHAR), subject-disjoint (MiliPoint)	overlapping windows leak	leakage audit (Ch. 8)
Validation composition	all windows of each recording	$\leq$55 windows per recording	one long recording dominated selection	Issue #16–#17
Optimiser	SGD; Adam	AdamW + one-cycle	standard for attention models; stable	training curves
Loss	plain cross-entropy; class weights	CE + label smoothing 0.1 (+KD)	near-duplicate windows; balanced classes	class counts (Ch. 3)
Regularisation	none / heavy augmentation	weight decay, dropout, early stopping, standard + optional robust augmentation	generalisation	augmentation ablation; robustness study
Model checkpoint	last epoch; best test	best validation macro-F1; exported seed chosen by validation	no test leakage in selection	audit: checkpoint consistency
Demo framework	Gradio, Flask	Streamlit + Plotly	interactive 3-D, built-in test harness	AppTest results (Ch. 13)
Point operations library	PyTorch Geometric, torch-cluster	pure PyTorch	no compiled extensions; portable	runs on CPU-only environment

Chapter 5

Data Pre-processing and Noise Handling

The pipeline (implemented in `src/data_radhar.py`, shared by both datasets) is described below, technique by technique, in the *what / why / how / effect / limitation / verification* format.

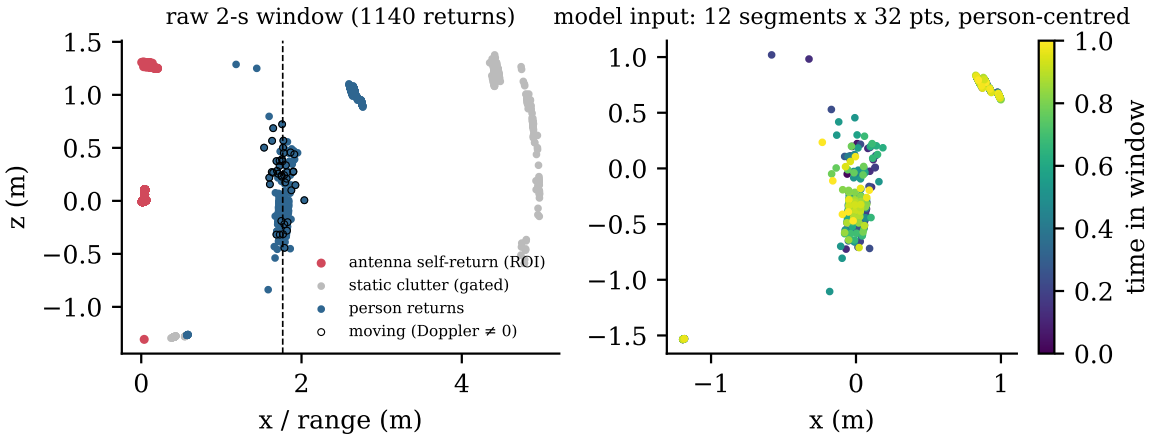


Figure 5.1: One raw 2-s RadHAR window (left), with antenna self-returns (red), static clutter removed by gating (grey), person returns (blue) and moving returns (black circles), and the resulting model input (right).

5.1 Windowing

What: 2-s windows (60 frames on RadHAR, 48 on MiliPoint) cut from one recording or single-action run. **Why:** single frames hold only 6–21 returns and no motion information, and 2 s matches the RadHAR protocol. **How:** `make_windows`; evaluation stride 10 frames (RadHAR) and 12 (MiliPoint); training stride 20 and 24, respectively. **Effect:** 3,565 RadHAR test windows (the original paper reports 3,538). **Limitation:** activities shorter than 2 s are not modelled. **Verification:** window counts per split in the leakage audit.

5.2 Region-of-interest (ROI) filter

What: drop returns with $x < 0.3$ m (RadHAR only). **Why:** 34.6% of all returns are antenna self-reflections at $x \approx 0$ (red points in Fig. 5.1). **How:** `roi_filter`. **Effect:** these returns no longer distort the centroid or consume sampling slots. **Limitation:** a person standing within 30 cm of the radar would be removed (an unrealistic set-up). **Verification:** EDA statistics.

5.3 Person centring with moving returns

What: express coordinates relative to the median of the window's *moving* returns (Doppler $\neq 0$); fall back to the median of all returns. **Why:** a plain centroid was pulled metres away by clutter (Issue #13). Static objects have zero radial velocity, whereas a performing person always produces some moving returns. The median is robust to the remaining outliers. **How:** `RadHARWindows.person_centre`. **Effect:** translation invariance with respect to where the person stands; removes distance as a cue. **Limitation:** a motionless person has no moving returns (fallback used); two moving people would be merged. **Verification:** ablation "no centring" was dropped for compute reasons. The gating ablation (below) covers the combined effect.

5.4 Spatial gating (clutter removal)

What: drop returns more than 1.2 m (horizontal distance) from the person centre. **Why:** after the ROI filter, 50.1% of returns are static clutter clusters at 2.5 m and 4.5 m. Because training and test recordings share the room, this clutter is also a potential scene *shortcut*. **How:** applied per segment before sampling. 1.2 m covers arm span and in-place walking (the median distance of returns from the centre is 0.9–1.6 m before gating). **Effect:** the sampled points come from the person. **Limitation:** returns from limbs farther than 1.2 m would be lost; the radius was chosen from EDA, not tuned on test data. **Verification:** ablation – *spatial gating* (Table 11.4).

5.5 Temporal segmentation and point sampling

What: split the window into $G = 12$ segments and sample $P = 32$ returns per segment (with replacement if fewer). **Why:** a fixed tensor size is needed for batching and edge inference; accumulating 4–5 frames per segment densifies the cloud while keeping temporal order. **How:** `__getitem__`; evaluation uses a fixed per-sample seed so that test inputs are deterministic. **Effect:** $12 \times 32 \times 6$ input. **Limitation:** duplicated points when segments are sparse. **Verification:** unit test of shapes and determinism.

5.6 Feature normalisation

Intensity is standardised with mean and standard deviation computed on the *training recordings only*. The validation and test sets reuse these values (verified by the audit, Chapter 8). Coordinates are already in metres and person-centred. Time is normalised to $[0, 1]$.

5.7 Training-time augmentation

Standard (all models): random rotation about the vertical axis ($\pm 15^\circ$), isotropic scaling ($[0.9, 1.1]$), 1-cm Gaussian jitter, and fresh point sampling every epoch. **Noise-robust training (optional, `-robust 1`):** per batch, each with probability 0.5: variable per-segment density (keep 30–100%), point dropout (0–20%), false-detection injection (0–10% of returns), and with probability 0.25 the loss of one temporal segment. All of these are applied to training batches only, from a separate random generator (seeded $10{,}000 + \text{seed}$) that is never used at test time.

5.8 Controlled noise conditions for evaluation

Measured dataset noise (self-returns, clutter, sparsity) is handled by the pre-processing above. To probe robustness beyond it, the *test* inputs are perturbed synthetically (`src/noise.py`), after pre-processing, with fixed seeds so that every model sees identical perturbed samples (Table 5.1). These conditions are experimental manipulations, not claims about the noise present in the datasets.

Table 5.1: Evaluation perturbations (applied to the held-out test set only).

Condition	Operation	Levels
Coordinate jitter	add $\mathcal{N}(0, \sigma^2)$ to x, y, z	$\sigma = 0.02, 0.05, 0.10$ m
Point dropout	remove returns (slots masked, variable count)	5, 15, 30%
Increased sparsity	keep a fraction of unique returns, re-sample to P	keep 50, 25, 12.5%
Outlier injection	replace returns by uniform false detections in ± 1.5 m	5, 10, 20%
Variable density	per-segment keep fraction $\sim U(k, 1)$	$k = 0.5, 0.25, 0.1$
Frame loss	drop whole temporal segments	25, 50% of segments
Combined	density + jitter + outliers + dropout	severity 1, 2, 3

5.9 Methods investigated but not used

Statistical and radius-based outlier removal and DBSCAN clustering were considered. Gating around the moving-point centre removes the dominant clutter with one interpretable parameter and no per-window clustering cost. Additional filters on the remaining sparse body returns would risk removing limb points, so they were not adopted. This decision was made from the EDA (the clutter forms far-away clusters) and was not tuned on test data.

Chapter 6

Model Selection and Architecture

6.1 Candidate architectures

PointNet [13]: $f(\{\mathbf{p}_i\}) = \gamma(\max_i h(\mathbf{p}_i))$, where h is a shared MLP and γ a classifier. The max is permutation-invariant. It is cheap, but no local geometry is modelled. **PointNet++** [14]: set-abstraction levels, each consisting of FPS sampling of n' centroids, a ball query (radius r, at most k neighbours) and a PointNet on every local group. **DGCNN** [15]: EdgeConv $\mathbf{h}'_i = \max_{j \in \mathcal{N}_k(i)} \phi(\mathbf{h}_j - \mathbf{h}_i, \mathbf{h}_i)$ over a k-NN graph that is recomputed in feature space at every layer. **Point Transformer** [16]: vector attention $\mathbf{y}_i = \sum_{j \in \mathcal{N}_k(i)} \rho(\gamma(\varphi(\mathbf{x}_i) - \psi(\mathbf{x}_j) + \delta_{ij})) \odot (\alpha(\mathbf{x}_j) + \delta_{ij})$ with relative position encoding $\delta_{ij} = \theta(\mathbf{p}_i - \mathbf{p}_j)$. **PointNet+BiLSTM**: a PointNet per temporal segment followed by a BiLSTM, as in recent radar HAR work [9]. **Voxel TD-CNN+BiLSTM**: the RadHAR baseline [4].

6.2 Selection rationale

The brief names a lightweight point transformer as the primary direction, provided the evidence supports it. Our design starts from the established parts, namely Transformer self-attention [27], relative-position attention on points [16] and PointNet-style pooling [13], and adapts them to the structure of the radar data:

1. Sparse frames but strong temporal order $\Rightarrow$ attention is *factorised*: first within short segments (spatial), then across segment tokens (temporal). This removes FPS and k-NN entirely.
2. Radar measures velocity $\Rightarrow$ an explicit per-segment motion descriptor is added to every token.
3. Edge cost $\Rightarrow$ width $d = 64$, 2 temporal blocks, distillation from a wider teacher, INT8 weights.

Established vs. project-specific. Multi-head self-attention, the pre-norm Transformer block, the [CLS] token, attentive/max pooling, knowledge distillation and per-channel weight quantisation are established techniques. Project-specific are: the factorisation into intra-segment and inter-segment attention for radar windows, the learned 3-D relative-position *bias* inside segments, the radar motion descriptor, and the moving-point centring and gating pre-processing. Every project-specific component is ablated in Table 11.4.

6.3 Layer-by-layer architecture of STPT-Lite

Table 6.1: STPT-Lite layers (batch B, $G = 12$, $P = 32$, $d = 64$, $H = 4$ heads, K classes). Parameter counts for $K = 5$.

Layer	Input	Output	Operation	Params
Point embedding	$BGP \times 6$	$BGP \times 64$	Linear–LN–GELU–Linear–LN–GELU	4,864
Intra-segment attention	$(BG) \times P \times 64$	same	pre-LN, QKV linear, $\mathrm{softmax}(QK^\top/\sqrt{16} + \beta(\Delta\mathbf{p}) + \log m)V$, output proj.	16,768
Relative-position bias β	$P \times P \times 3$	$P \times P \times 4$	MLP $3 \to 16 \to H$	132
Feed-forward	$(BG)P \times 64$	same	LN, Linear $64 \to 128$, GELU, Linear $128 \to 64$, residual	16,704
Segment pooling	$(BG) \times P \times 64$	$(BG) \times 64$	attentive pool $\|\|$ masked max $\to$ Linear $128 \to 64$	8,321
Motion descriptor	$B \times G \times 8$	$B \times G \times 64$	MLP $8 \to 32 \to 64$, added to tokens	2,464
[CLS] + positions	$B \times G \times 64$	$B \times 13 \times 64$	concatenate, add learned positions	896
Temporal block $\times 2$	$B \times 13 \times 64$	same	pre-LN MHSA ($H = 4$, key padding mask) + FFN ($2d$)	66,944
Head	$B \times 64$	$B \times K$	LN on [CLS], dropout 0.2, Linear	453
Total				117,546

The motion descriptor of segment g is $\mathbf{u}_g = [\bar{\mathbf{x}}_g,\ \bar{\mathbf{x}}_g - \bar{\mathbf{x}}_{g-1},\ \overline{|v|}_g,\ \log(1+n_g)]$, i.e. centroid (3), centroid displacement (3), mean absolute Doppler (1) and log point count (1).

6.4 Computational complexity

Joint self-attention over all $N = GP = 384$ points of a window costs $O(N^2 d) \approx 9.4$ M multiply-adds per layer. The factorised design costs $O(GP^2 d + G^2 d) \approx 0.8$ M + 0.01 M. Measured with

PyTorch's FLOP counter, the whole model needs 36.6 MFLOPs per 2-s window. PointNet++, DGCNN and Point Transformer additionally spend time on FPS loops and k-NN/ball queries, which are sequential or $O(N^2)$ (Table 11.1).

6.5 Training variants and final model selection

Three training variants of STPT-Lite were compared under identical settings: plain training, knowledge distillation from STPT-Large (KD), and KD combined with noise-robust augmentation (Section 5.7; `src/noise.py:robust_augment`: per batch, with probability 0.5 each, variable density with minimum keep 0.3, dropout up to 20%, false detections up to 10%, and with probability 0.25 the loss of one segment). Jitter and extreme sparsity are not used in training, so they remain unseen corruption types at test time. The final model was selected on *validation* macro-F1 averaged over seeds on both datasets (Issue #39): KD+robust had the highest mean validation macro-F1 on RadHAR and on MiliPoint, and it is the model that is exported, served by the demo and reported as "proposed". Test accuracy was not used for this choice.

6.6 Variants

Table 6.2: Model variants used in the experiments (parameters for 5 / 49 classes).

Model	Configuration	Params (5)	Params (49)
STPT-Large (teacher)	$d = 128$, 4 temporal blocks, 8 heads	725,262	730,938
STPT-Lite (proposed)	$d = 64$, 2 blocks, 4 heads	117,546	120,406
STPT-Tiny	$d = 32$, 2 blocks, 4 heads	30,682	32,134
PointNet	MLP 64-128-256, max-pool	76,229	81,905
PointNet+BiLSTM	per-segment PointNet 64-128, BiLSTM 64	109,125	114,801
PointNet++ (SSG)	SA(128, 0.3 m, 16) – SA(32, 0.6 m, 16) – global	413,381	424,689
DGCNN	3 EdgeConv ($k = 16$), 128 points	422,405	433,713
Point Transformer	2 PT blocks + transition down, 128 points	335,237	340,913
Voxel TD-CNN+BiLSTM	$10 \times 32 \times 32$ voxels, 3-D conv 8/16/16, BiLSTM 16	45,861	47,313

Chapter 7

Training and Optimisation

7.1 Environment

Table 7.1: Computational environment.

Hardware	2 vCPU (Intel Xeon), 7 GB RAM, no GPU (Linux cloud workspace)
Software	Python 3.13, PyTorch 2.15, NumPy 2.5, scikit-learn 1.9, SciPy 1.18, pandas 3.0, Matplotlib 3.11, Streamlit 1.67, Plotly 7.1
Parallelism	2 training jobs in parallel, 1 thread each (`scripts/runner.py`)
Latency measurement	batch 1, single thread, median of 200 runs after 20 warm-up runs

7.2 Hyper-parameters and their justification

All hyper-parameters are in `configs/default.yaml`. They were fixed *before* any test-set evaluation and are identical for every model, so the comparison is fair. Table 7.2 lists them.

Table 7.2: Training configuration.

Setting	Value	Reason
Optimiser	AdamW [28]	decoupled weight decay is the standard for Transformers; robust default for all models
Learning rate	2×10^{-3} peak	small models and batch 64; converged within a few epochs in smoke tests
Schedule	one-cycle, 10% warm-up	warm-up stabilises attention early; annealing to near zero gives a well-defined end point
Weight decay	0.05	regularisation for a small dataset
Batch size	64	CPU throughput vs. gradient noise
Loss	cross-entropy, label smoothing 0.1 [29]	avoids over-confident outputs on near-duplicate windows
Class weights	none	classes are nearly balanced (Chapter 3)

Setting	Value	Reason
Dropout	0.1 (attention/FFN), 0.2–0.5 (heads)	standard values for each architecture
Gradient clipping	global norm 1.0	prevents rare spikes from the attention softmax
Max epochs	15	upper bound only
Early stopping	patience 5 on validation macro-F1	stop when the model no longer generalises better
Checkpoint	best validation macro-F1	model selection on validation data only
KD	$\alpha = 0.7$, $T = 4$ [23]	common values; teacher = STPT-Large trained with the same seed
Seeds	0, 1, 2	three independent runs; mean $\pm$ sample std reported

7.3 Overfitting prevention

1. Early stopping and best-checkpoint saving on validation macro-F1.
2. One-cycle learning-rate scheduling.
3. Regularisation: weight decay, dropout, label smoothing.
4. Training-only augmentation (Chapter 5); validation and test inputs are deterministic.
5. Validation split by *recording* (RadHAR) or *subject* (MiliPoint), so selection reflects generalisation to unseen recordings or people.
6. Small models: the proposed model has 117.5 K parameters, compared with 4,773 RadHAR training windows and about 4,500 MiliPoint training windows.

For every epoch we log training loss, training accuracy, validation loss (NLL), validation accuracy, validation macro-F1 and the learning rate (`history.csv` in every run directory). The curves are discussed in Chapter 11.

7.4 Training-stability investigation

In the first round of RadHAR runs, results varied strongly across seeds (teacher: 95.5 / 96.1 / 87.0%) and the distilled student selected its epoch-1 checkpoint. The investigation (Issue #16–#17) traced this to validation composition. One 186-s squat recording contributed 553 of 2,709 validation windows (20%) and was predicted as “jump” throughout, so validation macro-F1

was dominated by a single recording. After capping each validation recording at 55 windows, validation and test accuracy agree (e.g. 94.5% validation accuracy at epoch 9 of the first re-run teacher). This is a genuine stability fix, not a tuning of results: the test set was not used to make it.

Chapter 8

Data Leakage Prevention

8.1 Splitting methodology

RadHAR: the official test recordings (58) are kept unchanged. The remaining 142 training recordings are split 80/20 *per class by recording* (113 train, 29 validation, fixed seed 0, independent of the model seed). **MiliPoint (primary)**: subject-disjoint, with subjects 8, 9, 10 for test, subjects 6, 7 for validation, and the remaining six subjects for training. Windows never cross the boundary of a single-action run. **Comparison protocols** (MiliPoint only, never used for model development): the chronological 80/10/10 cut of the MiliPoint code, and a random 80/10/10 split of overlapping windows.

8.2 Restrictions

- **Pre-processing**: the only data-dependent statistic, the intensity mean and standard deviation, is computed on training recordings and reused unchanged for validation and test. Gating radius and ROI threshold come from the EDA and are not learned.
- **Augmentation**: stochastic augmentation and robust-noise augmentation are applied only in training mode, from a separate random stream. Evaluation samples are deterministic (fixed per-index seed).
- **Test-set protection**: the checkpoint is chosen by validation macro-F1. The test split is evaluated once per run, after training. No hyper-parameter, pre-processing threshold or architecture choice was changed in response to a test score. The only data-handling change made after the first runs, the validation cap, was motivated by validation behaviour (Issue #17).
- **Exported model**: the seed exported for the demo is chosen by validation macro-F1, not by test accuracy.

8.3 Automated audit

`scripts/leakage_audit.py` recomputes every check from the data and run files. Its output (`docs/leakage_audit.md`) is reproduced in Table 8.1.

Table 8.1: Automated leakage-audit results.

Check	Result	Status
RadHAR recording overlap (train/val, train/test, val/test)	[0, 0, 0]	PASS
RadHAR identical-content recordings	0 groups	PASS
RadHAR identical frames ($\geq$5 pts) across splits	[0, 0, 0] of 167,610 frames	PASS
RadHAR windows of one recording in two splits	[0, 0, 0]	PASS
Intensity statistics fitted on train only	recomputed = used: True	PASS
RadHAR subject independence	no subject IDs in dataset	NOT VERIFIABLE
RadHAR session independence	test-to-train gap min 2 s, median 29 s; same 4 days	NOT INDEPENDENT
MiliPoint subject-disjoint: subject overlap; test frames inside training windows	none; 0 / 49,764 (0.0%)	PASS
MiliPoint chronological: subject overlap; test frames inside training windows	train/test: [0, 8]; 0 / 20,532 (0.0%)	SUBJECT OVERLAP
MiliPoint random windows: subject overlap; test frames inside training windows	n/a; 43,704 / 59,952 (72.9%)	LEAKY
Checkpoint = argmax validation macro-F1	91 / 91 runs	PASS

8.4 Findings

1. RadHAR splits share no recording, no identical recording content and no identical frame. Our splits are therefore recording-disjoint.

2. **RadHAR's official test set is not session-independent.** The ROS timestamps show that every test recording was made within 10 minutes of a training recording, on the same four days (median gap 29 s), and the dataset provides no subject IDs. Subject-independent generalisation therefore *cannot be verified* on RadHAR, and RadHAR accuracy should be read as "new recordings, same people and room".
3. On MiliPoint, the subject-disjoint split shares no subject and no frame between splits. This is the protocol that tests generalisation to new people, and it is our primary result.
4. The chronological protocol of the MiliPoint code shares subjects (0 and 8) between training and test.
5. The random-window protocol places 72.9% of test frames inside training windows, and 91.2% of test windows overlap a training window (by 58.7% of the window on average). Accuracies obtained this way measure memorisation of near-duplicates; Chapter 11 quantifies the inflation.
6. All completed runs satisfy "saved checkpoint = epoch of maximal validation macro-F1" (91 of 91 runs).

Chapter 9

Implementation Details

9.1 Repository structure

```
src/            radhar_parse.py  safe_pickle.py  data_radhar.py
    data_milipoint.py
                models.py  noise.py  train.py  inference.py
configs/        default.yaml
scripts/        runner.py  jobs_main.txt  eda.py  analyze.py  xai_robust
    .py
                leakage_audit.py  export_model.py  make_numbers.py
    run_all.sh
checkpoints/    exported inference bundles (model.pt, bundle.json,
    metrics.json)
demo/           app.py  samples/ (held-out test recordings)
tests/          test_pipeline.py (pytest)
results/        runs/<tag>/ (metrics.json, history.csv, probabilities,
    model.pt), summaries
docs/           DEVLOG.md  leakage_audit.md
paper/  report/ figures/
```

9.2 Module descriptions

Table 9.1: Modules: purpose, interfaces, error handling and verification.

Module	Purpose / key functions	Input	Output	Errors / verification
`radhar_parse`	`parse_text`: regex over ROS messages; frames from `point_id` resets	raw text	points $[N, 5]$, frame $[N]$, timestamps	`ValueError` if no records; unit tests

Module	Purpose / key functions	Input	Output	Errors / verification
`safe_pickle`	restricted `Unpickler.find_class`	.pkl path	Python objects	`UnpicklingError` for any non-numpy global; unit test with a malicious pickle
`data_radhar`	`split_recordings`, `make_windows`, `RadHARWindows` (ROI, centring, gating, sampling, augmentation)	recordings dict	tensors $x[G,P,6]$, mask, label	shape/determinism tests; audit
`data_milipoint`	run construction, axis remap, 3 split protocols	pkl files, `id.json`	same as above	subject-disjoint test; audit
`models`	9 model classes; `build`, `n_params`	x, mask	logits $[B,K]$	forward test with empty segments for every model
`noise`	7 seeded perturbations; `robust_augment`	x, mask, generator	new x, mask	determinism, no in-place change, effect-size tests
`train`	training loop, KD, early stopping, metrics, INT8, FLOPs, latency	CLI + YAML	run directory	smoke runs; audit of checkpoint selection
`inference`	`Bundle`: load, pre-process raw input, predict, predict with noise	bundle dir, raw file	probabilities, latency	rejects short/NaN/wrong-shape input; tests
`demo/app.py`	Streamlit UI	bundle, upload/sample	interactive views	invalid uploads shown as errors; AppTest

9.3 Key code

Factorised attention, segment tokenizer (`src/models.py`):

```
f = self.emb(x)                                                    # [N=B*G, P, d]
q, k, v = self.qkv(self.n1(f)).view(N, P, 3, h, d//h).permute(2, 0, 3,
    1, 4)
bias = self.rel(x[:, :, None, :3] - x[:, None, :, :3]).permute(0, 3, 1,
    2)
a = (q @ k.transpose(-1, -2)) / math.sqrt(d // h) + bias
a = a.masked_fill(m[:, None, None, :] == 0, -1e4).softmax(-1)
f = f + self.proj((a @ v).transpose(1, 2).reshape(N, P, d))
f = f + self.ff(self.n2(f))
```

Person centring and gating (`src/data_radhar.py`):

```
mov = allp[np.abs(allp[:, 3]) > 0]                                 # moving returns
     (Doppler != 0)
ctr = np.median(mov[:, :3] if len(mov) else allp[:, :3], 0)
keep = np.linalg.norm(p[:, :2] - ctr[:2], axis=1) <= self.gate
```

Knowledge-distillation loss (`src/train.py`):

```
kd = F.kl_div(F.log_softmax(logits / T, -1), F.softmax(tlog / T, -1),
              reduction='batchmean') * T ** 2
loss = (1 - alpha) * ce + alpha * kd
```

9.4 Reproducibility

Every run fixes `torch.manual_seed`, NumPy's seed and the data-loader generator. Evaluation inputs use a per-index seed. Each `metrics.json` stores the full argument list, the YAML configuration and the PyTorch version. `scripts/run_all.sh` reproduces the full pipeline end to end.

Chapter 10

Failed Experiments, Errors and Rectification History

This chapter reproduces the continuous issue-resolution log (`docs/DEVLOG.md`). Entries are taken from the actual commands, error messages, logs and results of the development; no entry was invented. Times are UTC on 11–12 November 2026.

Table 10.1: Issue-resolution log.

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
1	Environment	No GPU; 2 CPU cores, 7 GB RAM	`nvidia-smi` not found, `nproc`=2	CPU-only design: lightweight models, reduced training stride/epochs, 2 parallel jobs	timing benchmarks (#12)	all experiments run on CPU
2	Dataset	RadHAR obtained	`git clone nesl/RadHAR` (975 MB, 200 raw txt files)	—	file count 200	OK
3	Dataset	MiliPoint download blocked	Google Drive returns proxy 403 from the development environment	`MiliPoint_data.zip` downloaded manually through a browser and copied into the project	zip 159,922,979 B, 19 pkl + action_label.npy	OK
4	Environment	`pip install torch -index-url download.pytorch.org` failed ("no matching distribution")	download.pytorch.org is not on the network allow-list	Installed torch 2.15 from PyPI	`import torch`	OK
5	Parsing	RadHAR raw files are ROS text dumps (one point per message)	README format section	Regex parser; frame index = cumulative count of `point_id: 0`	durations 71.5 / 21.4 min match the official 71.6 / 21.4 min; median 21 pts/frame	OK
6	Security	MiliPoint data are Python pickles (arbitrary code execution risk)	`pickletools` scan: only 3 STACK_GLOBAL opcodes (numpy reconstruct)	`src/safe_pickle.py` restricted unpickler allowing only numpy array reconstruction	all 19 files load	OK

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
7	Dataset	MiliPoint axes differ from RadHAR	keypoints: nose z=+0.62, ankles z=-0.87, mean y=1.5 m → y is forward	remap (x,y,z)→(y,x,z)	cross-dataset transfer uses one frame	OK
8	Dataset	Verified label count	—	—	212,920 labelled frames = value in MiliPoint paper	OK
9	Training	`nn.TransformerEncoder` fast path crashed in eval (`'function' object has no attribute 'device'`)	PyTorch fast-path check with GELU activation callable	Own pre-norm Transformer block	smoke run completes	OK
10	Results	2-epoch smoke test: STPT-Lite 93.1 % RadHAR test	—	—	—	pipeline works end to end
11	Deployment	Dynamic INT8 (torch.ao) dropped accuracy 93.1 → 85.8 % and was slower (3.0 vs 2.1 ms)	per-tensor activation quantisation on tiny layers; quant/dequant overhead	Weight-only per-channel INT8	accuracy 91.56 → 91.61 %, 0.49 → 0.135 MB	OK
12	Compute	Heavy baselines 5-10x slower than proposed (PointNet++ 826 ms, Point Transformer 1083 ms, voxel 2061 ms per batch of 64)	$O(N^2)$ kNN / FPS loops on CPU; 3-D convolutions	training stride 20 (RadHAR) / 24 (MiliPoint), $\leq$15 epochs + early stopping, DGCNN/PT on 128-point subsets, voxel CNN 8/16/16 filters	per-run times in metrics.json	same budget for every model
13	Pre-processing	Window centroid pulled away from the person	EDA: static clutter clusters at 2.5 m and 4.5 m; moving (Doppler $\neq 0$) returns only on the person (range histograms)	Person centre = median of moving returns; 1.2 m horizontal gating	50.1 % of post-ROI returns removed, all far from person (Fig. preproc)	runs restarted
14	Training	`nn.MultiheadAttention` fast path crashed after quantisation	fast path again	Own attention (qkv + proj Linear)	smoke run OK	OK

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
15	Literature	Published MiliPoint accuracies 34.45 % (authors) vs 98.25 % (MAK-GCN, random 80/10/10)	split protocol	Added subject-disjoint (primary), chronological and leaky random protocols	protocol experiment	queued
16	Validation	Val accuracy ∼79 % vs test ∼96 %	per-recording val accuracy: `___squats_186s.txt` (553 windows = 20 % of val) predicted "jump" 92 % of the time; all other recordings 71-100 %	—	—	led to #17
17	Model selection	STPT-Lite+KD selected epoch 1 (90.7 %); teacher seeds 95.5/96.1/87.0 %	one long outlier recording dominates validation macro-F1	Cap each validation recording at 55 evenly spaced windows (= a standard 20-s recording)	val = 1,595 windows, outlier weight 20 % → 3.4 %	RadHAR runs restarted
18	Specification	Consolidated master specification received	requires noise-robustness study, train/val loss+acc+LR curves, macro P/R, leakage audit, config files, inference script, Streamlit demo, technical report, PPT + notes, QA matrix	queue paused before further compute; training-side changes made first (#19-)	—	—
19	Noise robustness	Spec makes robustness a core requirement; no noise module existed (only train-time jitter)	gap analysis	`src/noise.py`: 7 test perturbations x 2-3 severities (jitter, point dropout, sparsity, outlier injection, variable density, frame loss, combined) with explicit seeded generators; `robust_augment` for training	smoke run with `-robust 1`	queued: STPT-Lite+KD+robust and Point-Net+robust, 3 seeds, both datasets

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
20	Training logs	History lacked train accuracy, validation loss and LR; metrics lacked macro precision/recall and per-class P/R	spec section 8.2 / 9	train.py logs train_loss/acc, val_loss (NLL)/acc/F1, lr per epoch; metrics add macro-P/R, per-class P/R/F1/support, config, torch version	smoke run history.csv	OK
21	Reproducibility	Hyper-parameters only as argparse defaults	spec section 10	`configs/default.yaml` read by train.py (flags override)	smoke run	OK
22	Ablation	No ablation of augmentation components	spec 9.2	`-aug none` / `nojitter` jobs (3 seeds)	—	queued
23	Process	RadHAR teacher seeds 0/1 from the 13:36 queue deleted before restart (trained before metric/logging changes)	consistency: all reported runs must share one code version	queue restarted 13:41	runner log	—
24	Leakage audit	Automated audit written (`scripts/leakage_audit.py`)	—	checks recording/-subject/session overlap, identical recordings & frames, frame sharing between windows, train-only statistics, checkpoint selection	`docs/leakage_audit.md`	RadHAR: 0 overlaps, 0 identical frames, train-only stats verified
25	Leakage audit	RadHAR official test set is NOT session-independent	ROS timestamps: every test recording lies within 10 min of a training recording (median gap 29 s), same 4 recording days; no subject IDs exist	cannot be corrected without changing the official benchmark; reported as a limitation; MiliPoint subject-disjoint split made the primary evaluation	audit table	limitation stated

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
26	Leakage audit	MiliPoint chronological (official-code) split shares subjects 0 and 8 between train and test; random split: 72.9 % of test frames lie inside training windows, 91.2 % of test windows overlap a training window (mean 58.7 %)	audit	both kept only as comparison protocols	audit table	quantifies inflation of published numbers
27	Inference	`src/inference.py` — (Bundle: load model + bundle.json, same pre-processing as training via RadHARWindows) + `scripts/export_model.py` (seed chosen by validation macro-F1)	—	—	CLI on a raw RadHAR test file: per-window + majority predictions; garbage file → `ValueError: no RadHAR point records found`	OK
28	Demo	Streamlit AppTest: `StreamlitDuplicateElementId` for plotly charts	identical auto-generated element IDs	unique `key` per chart	AppTest: 0 exceptions	OK
29	Demo	First single-window latency 96.6 ms in the app	first call allocates; measured without warm-up	warm-up call + median of 20 runs	—	OK
30	Demo	End-to-end AppTest on smoke bundle	—	—	0 exceptions for all 7 perturbations and 3 samples; boxing/jack-/walk predicted = ground truth	OK (final bundle exported after training)

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
31	Planning	Full experiment plan needs ∼9-10 h on the 2-core CPU	per-epoch timings of the first runs	Prioritised queue (112 jobs): DGCNN and Point Transformer (the two most expensive models) run with 1 seed on MiliPoint; RadHAR→MiliPoint transfer and minor ablations (segment count, ROI-only, no-centring) and MiliPoint ablations dropped	—	reported in the paper/report as CPU compute cost
32	Delivery	GitHub repository required as a deliverable	team request	public repo under the team lead's account at the end; code, configs, tests, demo, exported models, result summaries, figures, paper/report/slides; raw datasets linked, not copied (1.1 GB, GitHub 100 MB file limit)	push verified by listing the remote	planned
33	Training (Mili-Point)	First 3 MiliPoint runs (teacher s0/s1, STPT-Lite s0) selected epoch 15/15 or 14/15; validation macro-F1 still rising (teacher s0: 0.387 → 0.413 → 0.416 over epochs 13-15)	15-epoch cap stops the harder 49-class task before convergence (RadHAR: only 2 of 24 runs at the cap, val saturated)	MiliPoint cap raised to 30 epochs for every model and protocol (early stopping unchanged); the 3 runs moved to `results/superseded/` and retrained	best-epoch distribution of the new runs	queue restarted 15:31

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
34	Planning	30-epoch MiliPoint training roughly doubles MiliPoint cost; projected queue end too late for the remaining evaluation and documentation	measured epoch times	Removed: DGCNN on MiliPoint; DGCNN seeds 1-2 and voxel seed 2 on RadHAR; PointNet++ and PointNet seeds 1-2 in the protocol study; seed 2 of the transfer study. Kept 3 seeds for all STPT models, PointNet, PointNet-BiLSTM, PointNet++ and Point Transformer (RadHAR), all ablations	final seed counts listed in every results table	reported as CPU compute cost
35	Planning	Measured throughput after the MiliPoint change (∼5-6 runs/h) projects queue end at ∼23:00 UTC	runner log	Ablations and protocol study reduced to 2 seeds (0, 1)	seed counts in tables	—
36	Infrastructure	Background runner killed at the 2-h limit of the execution environment; 2 in-progress runs lost	runner log	runner stops launching jobs after 80 min and exits after running jobs finish; restarted by the operator	runner log 'TIME BUDGET'	no further partial runs
37	Infrastructure	Draining runner still killed at the 2-h limit while waiting for MiliPoint Point Transformer (>48 min run); run lost	single run longer than the remaining window	MiliPoint Point Transformer moved to the optional queue; launch window kept at 80 min; runner instances coordinate via a 'log modified < 15 min' busy check	runner log	—

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
38	Training	Voxel TD-CNN run killed by the out-of-memory killer (exit 137, 3.6 GB RSS) at the first validation pass	evaluation batch of 256 windows = 3,072 voxel grids through 3-D convolutions, while two other processes were running	evaluation batch 32 for the voxel model; robustness evaluation batch 128	dmesg OOM record; rerun	rerun queued
39	Model selection	Final configuration chosen among STPT variants	mean validation macro-F1 over 3 seeds: RadHAR KD+robust 95.74 vs KD 95.14 vs Tiny-KD 95.31; MiliPoint KD+robust 53.52 vs KD 53.05 (test sets not consulted)	STPT-Lite + KD + noise-robust training designated the proposed/deployed model; statistics use it as reference; exported seed chosen by validation	export_model.py records the selection rule	—
40	Demo	Final bundles exported (RadHAR seed 0, MiliPoint seed 1, chosen by validation macro-F1); AppTest with final model: 0 exceptions over 7 perturbations and 3 samples, predictions = ground truth; screen recording failed to switch the perturbation dropdown	Playwright located BaseWeb select by text, which did not match	locate the combobox by its accessible name 'Perturbation'	frame grid of demo.mp4 shows outliers/sparsity/combined states	demo.mp4 (2:09)
41	Statistics	Draft text claimed baseline differences on MiliPoint significant at p < 1e-50	re-reading results/stats.json: largest Holm-corrected p over seeds and baselines is 6.0e-47 (PointNet, seed 2)	claim corrected to p < 1e-45 in paper and report	value re-checked against stats.json	over-claim removed
42	Paper	Introduction claimed cross-dataset transfer "in both directions"	only MiliPoint→RadHAR transfer was run (jobs_main.txt)	wording changed to MiliPoint→RadHAR transfer	grep of paper and report sources	over-claim removed

#	Stage	Issue / decision	Root cause / evidence	Correction	Verification	Outcome
43	Ablation	RadHAR ablation uninformative: removing components changed accuracy by -1.0 to +1.7 points, all within the seed spread of the full model (±1.43)	RadHAR saturated at 95–98 % for all models	MiliPoint (unseen subjects) ablation of attention, motion descriptor and temporal Transformer added (scripts/jobs_final.txt)	tab_ablation lists both datasets with seed counts	components judged on the informative benchmark
44	Robustness	Robustness figures showed accuracy recovering at higher severities for jitter, outliers, frame loss and combined	plotting code sorted levels descending for these conditions while tick labels followed noise.CONDITIONS (mild→severe); CSV values were correct	points indexed by the exact level list used for the labels; distinct colours per baseline	figure re-checked against robustness_radhar.csv (e.g. voxel jitter 0.1 = 81.5 %)	figures corrected before inclusion
45	Paper	MiliPoint test-set size stated as 3,569 windows	test_labels.npy of every subject-split run has 3,472 windows (matches leakage audit)	text and table caption corrected	value read from run files	consistent
46	Ablation	Single-seed variants compared against the 3-seed mean of the full model, which mixed seed effects into the difference	MiliPoint full model varies 38.6–50.3 % across seeds	differences now computed against the full model trained with the same seeds; single runs marked	tab_ablation regenerated; motion descriptor shows no measurable gain and is reported as such	paired comparison

10.1 Most consequential issues

Clutter-biased centring (#13). Symptom: in the exploratory plots, the "centred" model input placed the person about 0.7 m off-centre, together with a large second cluster. Diagnosis: range histograms of all returns vs. moving returns showed that static clusters at 2.5 m and 4.5 m carried about half of the returns, while all moving returns were on the person. Alternatives considered: DBSCAN clustering (per-window cost, parameter sensitivity) and radius/statistical outlier removal (removes sparse limb returns). Correction: centring on moving returns plus 1.2 m gating. Verification: the before/after figure and the ablation – *spatial gating*. All runs were restarted so that every reported number uses the same pipeline.

Validation dominated by one recording (#16–#17). Symptom: validation accuracy (79%) far below test accuracy (96%), a seed spread of 9 points and an epoch-1 checkpoint for one model. Diagnosis: per-recording validation accuracy showed a single 186-s squat recording (20% of validation windows) predicted as "jump". Correction: at most 55 evenly spaced windows per validation recording. Verification: validation and test accuracy now agree. The test set played no part in the fix.

Quantisation (#11). PyTorch dynamic INT8 quantisation lost 7 points of accuracy and was *slower* for these tiny layers. Weight-only per-channel INT8 kept accuracy (within 0.05 points) and reduced storage by a factor of 3.6. Only the INT8 storage size is claimed, not an INT8 speed-up.

Leakage findings (#24–#26). The audit did not reveal leakage in our own protocols. It did reveal that the RadHAR benchmark cannot test session or subject independence, and that the random-window protocol used in some prior work leaks most of its test frames into training. This changed the emphasis of the evaluation (MiliPoint subject-disjoint as the primary result) and became a contribution of the paper.

Chapter 11

Experimental Results

All numbers in this chapter are generated by `scripts/analyze.py` and `scripts/make_numbers.py` from the run directories, so they cannot drift from the logs. Means and standard deviations are over the seeds stated in each table. The test sets were evaluated once per run, with the checkpoint chosen on validation data.

11.1 Main comparison

Table 11.1: RadHAR official test recordings (3,565 windows, 5 classes). Mean $\pm$ std over 3 seeds. Sig.: number of seeds in which the Holm-corrected McNemar test against the proposed model gives $p < 0.05$; $\uparrow$ ours better / $\downarrow$ ours worse. Latency: median, batch 1, one CPU thread.

Model	Params (K)	MFLOPs	Lat. (ms)	Accuracy (%)	Macro-F1 (%)	Sig.
Voxel TD-CNN+BiLSTM [RadHAR]	45.9	180.5	6.60	95.01 ± 0.16	95.03 ± 0.17	2/3↑
PointNet	76.2	31.8	0.74	96.31 ± 0.66	96.36 ± 0.64	1/3↑
PointNet+BiLSTM	109.1	6.6	0.83	96.14 ± 0.23	96.16 ± 0.21	2/3↑
PointNet++ (SSG)	413.4	133.1	8.44	97.34 ± 0.18	97.37 ± 0.19	1/3↓
DGCNN	422.4	137.9	6.95	95.85 ± 0.00	95.89 ± 0.00	1/3↑
Point Transformer	335.2	264.4	7.98	97.01 ± 0.25	97.04 ± 0.24	1/3↑
STPT-Large (teacher)	725.3	139.5	3.59	96.44 ± 1.43	96.48 ± 1.42	1/3↑
STPT-Lite	117.5	36.6	2.14	96.22 ± 1.43	96.27 ± 1.41	2/3↑
STPT-Lite + KD	117.5	36.6	2.08	96.04 ± 0.78	96.08 ± 0.76	1/3↑
STPT-Tiny + KD	30.7	12.1	1.92	96.86 ± 0.17	96.90 ± 0.16	1/3↑
STPT-Lite + KD + robust (ours)	117.5	36.6	2.58	96.83 ± 0.69	96.88 ± 0.68	–
PointNet + noise-robust training	76.2	31.8	0.73	$\mathbf{97.38 \pm 0.11}$	$\mathbf{97.40 \pm 0.11}$	1/3↓

Table 11.2: MiliPoint, subject-disjoint test (subjects 8–10, 3,472 windows, 49 actions). Same conventions as Table 11.1.

Model	Params (K)	MFLOPs	Lat. (ms)	Accuracy (%)	Macro-F1 (%)	Top-3	Sig.
PointNet	81.9	31.8	0.74	32.57 ± 4.59	29.72 ± 4.50	59.5	3/3↑
PointNet+BiLSTM	114.8	6.6	0.74	28.11 ± 4.06	28.09 ± 4.66	56.1	3/3↑
PointNet++ (SSG)	424.7	133.1	7.94	8.07 ± 5.47	5.12 ± 5.76	21.4	3/3↑
STPT-Large (teacher)	730.9	139.5	3.65	$\mathbf{50.39 \pm 1.39}$	$\mathbf{50.84 \pm 2.11}$	75.8	1/3↑
STPT-Lite	120.4	36.6	2.60	45.79 ± 6.30	45.95 ± 7.05	73.9	1/3↑
STPT-Lite + KD	120.4	36.6	2.07	48.55 ± 4.35	47.62 ± 6.08	76.4	1/3↓
STPT-Lite + KD + robust (ours)	120.4	36.6	2.00	49.56 ± 1.98	48.80 ± 1.68	78.0	–
PointNet + noise-robust training	81.9	31.8	0.78	18.66 ± 8.83	15.67 ± 8.98	40.9	3/3↑

MiliPoint, unseen subjects (Table 11.2). The STPT family clearly outperforms all re-implemented baselines. The proposed STPT-Lite+KD+robust (selected on validation macro-F1, Issue #39) reaches $49.56 \pm 1.98\%$ top-1 (78.03% top-3); distillation alone gives $48.55 \pm 4.35\%$.

This compares with $32.57 \pm 4.59\%$ for the best baseline (PointNet). Every pairwise difference to a baseline is significant in all three seeds (exact McNemar, Holm-corrected, $p < 10^{-45}$). Distillation raised the mean and, above all, removed a failed seed (STPT-Lite without KD: one seed at 38.6%).

RadHAR (Table 11.1). All point-based models lie between about 95 and 98%, and differences between them are mostly within one standard deviation. The proposed model reaches $96.83 \pm 0.69\%$. PointNet++ is the most accurate here, at 4–5× our latency and 3.5× our parameter count. Every model, including the voxel CNN-LSTM from the original paper, exceeds the published 90.47%, which shows the effect of the shared pre-processing.

11.2 Per-class results and confusion

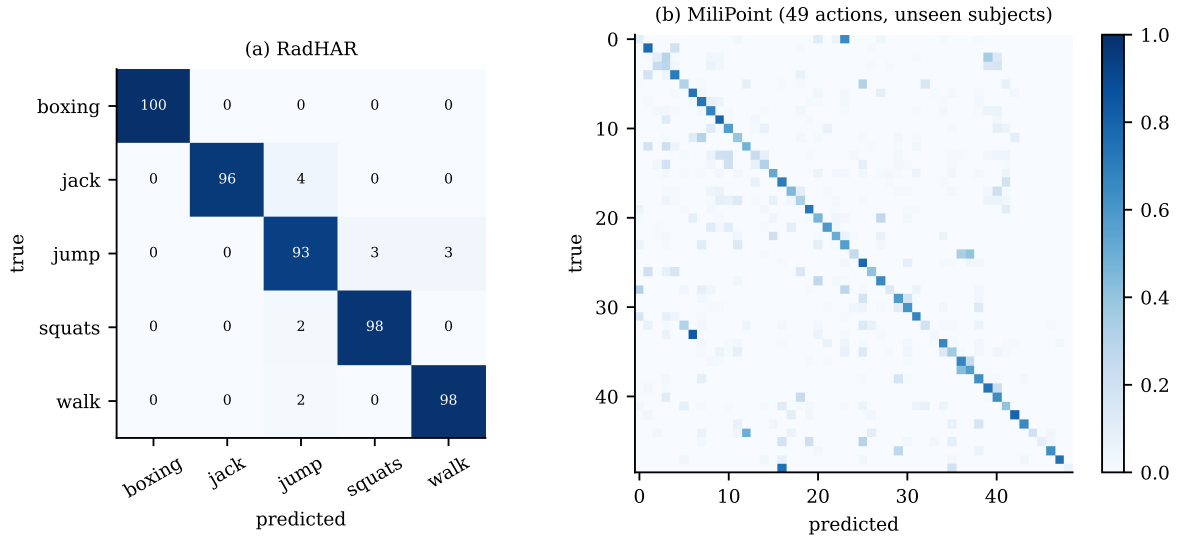


Figure 11.1: Row-normalised confusion matrices of the proposed STPT-Lite+KD+robust (seed 0).

Table 11.3: Per-class test precision, recall and F1 (%) of the proposed model (STPT-Lite+KD+robust) on RadHAR (mean over seeds).

Class	Precision	Recall	F1	Windows
boxing	99.86	99.91	99.88	714
jumping jacks	99.69	95.15	97.36	673
jump	91.01	94.41	92.66	734
squats	97.57	96.79	97.17	674
walk	96.83	97.79	97.31	770

On MiliPoint the per-action recall of the proposed model (mean over seeds) ranges from 6.4% (action 33) to 83.9% (action 9); 27 of 49 actions are recalled at $\geq$50% and 6 below

20%. The weakest actions are confused with kinematically similar ones (off-diagonal clusters in Fig. 11.1).

11.3 Training and validation behaviour

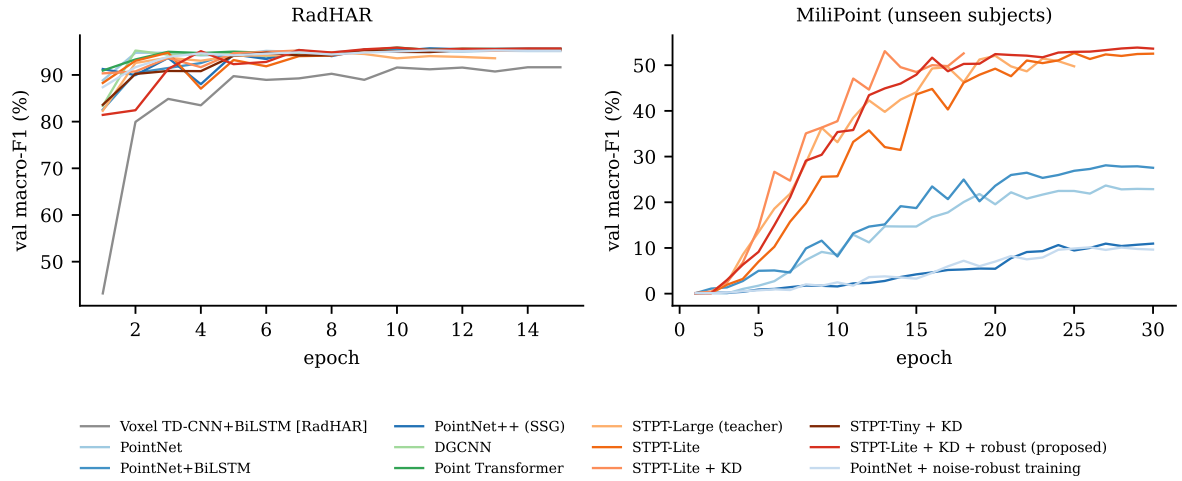


Figure 11.2: Validation macro-F1 per epoch (seed 0) for every model. RadHAR saturates within a few epochs; on MiliPoint the STPT models converge between epochs 15 and 30, while the PointNet family keeps a large gap.

Training/validation loss, accuracy and learning rate for every epoch of every run are stored in `results/runs/*/history.csv`. On RadHAR, 32 of the 33 main-comparison runs peaked before the 15-epoch cap. On MiliPoint the 30-epoch cap was introduced after the first runs peaked at the 15-epoch cap (Issue #33). After the change, the STPT runs peaked between epochs 13 and 29.

11.4 Ablation, protocol study and transfer

MiliPoint (unseen people) is the informative benchmark for the architecture. Compared with the full model trained with the same two seeds (49.4% on average), removing the temporal Transformer (mean over segment tokens instead) costs 11.9 points, and removing the intra-segment attention (pooling only) costs 6.5 points. Both losses hold in each seed separately. Fine-grained actions are therefore distinguished both by how the returns of a short segment relate to each other and by how the segments evolve over the window. The explicit motion descriptor brings no measurable gain (+0.5 points on average, +2.8 and −1.8 in the two seeds): the attention layers apparently recover centroid and displacement information from the point coordinates themselves. We keep the descriptor because it costs only 2.5 K parameters (2% of the model), but we do not claim it as a contribution to accuracy. **RadHAR** cannot rank components. Every variant, including removing Doppler/intensity, gating or augmentation, lies

Table 11.4: Ablation of STPT-Lite (no distillation): test accuracy (%), mean ± std over seeds (variants: 2 on RadHAR, 2 on MiliPoint; full model: 3), change w.r.t. the full model trained with the same seeds in brackets; † single run.

Variant	RadHAR	MiliPoint (subj.)
STPT-Lite (full, no KD)	96.22 ± 1.43	45.79 ± 6.30
– intra-segment attention (max+attentive pool only)	96.90 ± 0.18 (+0.93)	42.87 ± 3.03 (-6.52)
– motion descriptor	97.67 ± 0.16 (+1.70)	49.90 ± 2.06 (+0.50)
– temporal Transformer (mean over segments)	96.70 ± 0.61 (+0.73)	37.54 ± 6.62 (-11.85)
– Doppler & intensity (*xyz* only)	96.42 ± 1.69 (+0.45)	–
– spatial gating	97.62 ± 0.40 (+1.64)	–
– all training augmentation	96.68 ± 0.54 (+0.70)	–
– coordinate jitter augmentation	95.86 ± 1.13 (-0.11)	–

Table 11.5: MiliPoint (49 actions) top-1 accuracy (%) of the same models under three splits. Published numbers: 34.45 (PointNet++, [1]) and 98.25 (MAK-GCN, random 80/10/10, [2]).

Model	Subject-disjoint (ours, primary)	Chronological (MiliPoint code)	Random windows (leaky)
STPT-Lite	45.79 ± 6.30	58.38 (n=1)	79.48 (n=1)
PointNet	32.57 ± 4.59	32.42 (n=1)	44.44 (n=1)

within about 1.7 points of the full model, inside its seed spread (± 1.4), because every model already exceeds 95%. The effect of the pre-processing is visible across models instead: with the same gating and centring, all baselines exceed the published 90.47%.

Pre-training on MiliPoint helps most when little RadHAR data is available. With 10% of the RadHAR training recordings it raises test accuracy from 61.22% to 78.98% (two seeds; +18 points, larger than the seed spread). With 25% the gain shrinks to about one point, inside the seed spread, and with all training data the pre-trained and scratch models are equivalent (95.23% vs. 96.42%). Since the two datasets differ in radar, frame rate, subjects, room and label set, this suggests that the segment tokenizer learns largely sensor-independent motion features from MiliPoint's 49 actions, which are most valuable when target labels are scarce, as in a new home with few annotated recordings. With all data, one pre-trained seed reached its best validation score after the first epoch and was stopped early (93.8%); we report it unchanged.

11.5 Noise robustness

Fig. 11.3 reports test accuracy under seven perturbation types, three severities each, applied with fixed seeds so that every model sees the same perturbed windows. **MiliPoint (unseen**

Table 11.6: Cross-dataset transfer (STPT-Lite, *xyz* input): test accuracy (%) when fine-tuning on a fraction of target training recordings/runs, from scratch vs. initialised from the MiliPoint model (mean ± std over 2 seeds; n=1 marks a single run). Scratch at 100% is the *xyz*-only ablation run.

Direction	Target data	Scratch	Pre-trained
MiliPoint → RadHAR	10%	61.22 ± 0.58	78.98 ± 4.34
MiliPoint → RadHAR	25%	86.10 ± 1.37	87.45 ± 2.32
MiliPoint → RadHAR	100%	96.42 ± 1.69	95.23 ± 2.02

Table 11.7: Robustness on MiliPoint (unseen subjects): test accuracy (%) / prediction stability (% of predictions unchanged w.r.t. clean), mean over seeds.

Condition (level)	PointNet	PointNet+rob.	PointNet++	STPT+KD	STPT+KD+rob.
clean (0)	32.6 / 100	18.7 / 100	14.2 / 100	48.5 / 100	49.6 / 100
jitter (0.05)	26.3 / 62	15.7 / 66	11.0 / 31	39.6 / 69	42.0 / 74
dropout (0.3)	31.6 / 73	19.0 / 69	13.1 / 53	46.3 / 77	47.3 / 80
sparsity (0.25)	22.5 / 35	14.8 / 29	8.4 / 20	24.0 / 29	29.4 / 38
outliers (0.1)	21.6 / 42	16.5 / 63	8.8 / 21	41.9 / 70	45.0 / 77
density (0.25)	28.9 / 57	18.2 / 50	11.3 / 38	40.1 / 58	43.3 / 64
frameloss (0.5)	24.7 / 44	15.8 / 38	11.6 / 30	37.6 / 59	39.0 / 61
combined (2)	17.9 / 30	13.8 / 36	8.2 / 14	29.9 / 44	33.0 / 51

people). The proposed model stays the most accurate under every condition except the most extreme sparsity. Noise-robust training improves STPT-Lite+KD under all perturbations, by 0.9–5.4 points (e.g. 20% false detections: 34.6→39.8%; density 0.1: 35.6→39.6%), including jitter, which was not used in training (22.8 vs. 20.5% at $\sigma = 10$ cm). At 12.5% of the returns kept (about one point per frame), STPT-Lite (13.9%) falls to the level of PointNet (14.7%): with almost no points per segment, intra-segment attention has nothing to attend to. **RadHAR.** Dropout, density changes and jitter up to 5 cm cost every point-based model at most 2.5 points. Losing half of the segments costs DGCNN and Point Transformer about five points but the proposed model only 1.2 points (96.8→95.6%), which matches the even temporal attention in Section 11.6. Two conditions separate the models. Under extreme sparsity, Point Transformer (92.4%) and PointNet with robust training (91.1%) degrade least, the proposed model reaches 80.0%, and STPT-Lite+KD without robust training drops to 69.7%. False detections are the main weakness of the STPT family on RadHAR: at 20% injected outliers the proposed model keeps 55.1% (39.3% without robust training), whereas PointNet with the same robust training keeps 96.0% and the voxel CNN-LSTM 93.7%. A likely cause is that outliers corrupt the motion descriptor (centroid, mean Doppler) and receive attention weight, while a max-pooled PointNet with robust training learns to ignore isolated points and voxel occupancy dilutes them.

Table 11.8: Robustness on RadHAR: test accuracy (%) / prediction stability (% of predictions unchanged w.r.t. clean), mean over seeds.

Condition (level)	PointNet	PointNet+rob.	PointNet++	STPT+KD	STPT+KD+rob.
clean (0)	96.3 / 100	97.4 / 100	97.5 / 100	96.0 / 100	96.8 / 100
jitter (0.05)	96.2 / 98	96.9 / 99	97.0 / 99	95.6 / 99	96.7 / 99
dropout (0.3)	95.8 / 99	97.1 / 99	96.9 / 99	95.5 / 99	96.4 / 99
sparsity (0.25)	92.8 / 96	95.0 / 97	93.8 / 96	89.7 / 92	93.0 / 95
outliers (0.1)	57.1 / 56	96.6 / 99	57.0 / 57	59.9 / 60	72.5 / 72
density (0.25)	95.1 / 98	96.5 / 99	96.4 / 98	94.2 / 98	95.8 / 98
frameloss (0.5)	93.7 / 96	95.2 / 97	94.6 / 96	94.6 / 97	95.6 / 98
combined (2)	56.0 / 55	95.5 / 98	57.5 / 57	60.5 / 61	72.3 / 72

Robust training therefore helps, but it does not close this gap. A robust centroid (median) in the motion descriptor and outlier-aware attention are the obvious next steps.

11.6 Explainability

Temporal attention and occlusion. The [CLS] attention of the last block is spread almost evenly over the twelve segments (0.07–0.09 per segment, against 0.083 for uniform attention), with class-specific emphasis: the middle of the window for jumping jacks, early segments for squats, and the first segment for walking (Fig. 11.4). Removing a single segment (occlusion) lowers the true-class probability by less than one percentage point on average. The model therefore integrates evidence over the whole window and does not depend on any single 170-ms segment, which is consistent with its tolerance to frame loss. Attention and occlusion importance agree only moderately (Pearson $r = 0.49$ over the class$\times$segment cells), so we treat attention as a coarse indication and not as a faithful explanation. **Point saliency.** Gradient saliency concentrates on the returns of the moving limbs and torso and is low on isolated returns far from the body, i.e. on the residual clutter that survives gating. **Embeddings.** In a t-SNE of RadHAR test embeddings the classes form compact, well-separated clusters (silhouette 0.82 in the model's embedding space versus 0.24 for PointNet). **Errors.** The remaining RadHAR errors are mostly windows predicted as *jump* (precision 91%), which shares the strong vertical Doppler pattern of jumping jacks and squats. On MiliPoint, errors concentrate in groups of kinematically similar actions.

11.7 Computational efficiency

STPT-Lite needs 36.6 MFLOPs and 2.58 ms per 2-s window on one CPU thread. It occupies 0.49 MB in FP32 and 0.14 MB with INT8 weights, and INT8 accuracy is unchanged (96.82% RadHAR, 49.59% MiliPoint).

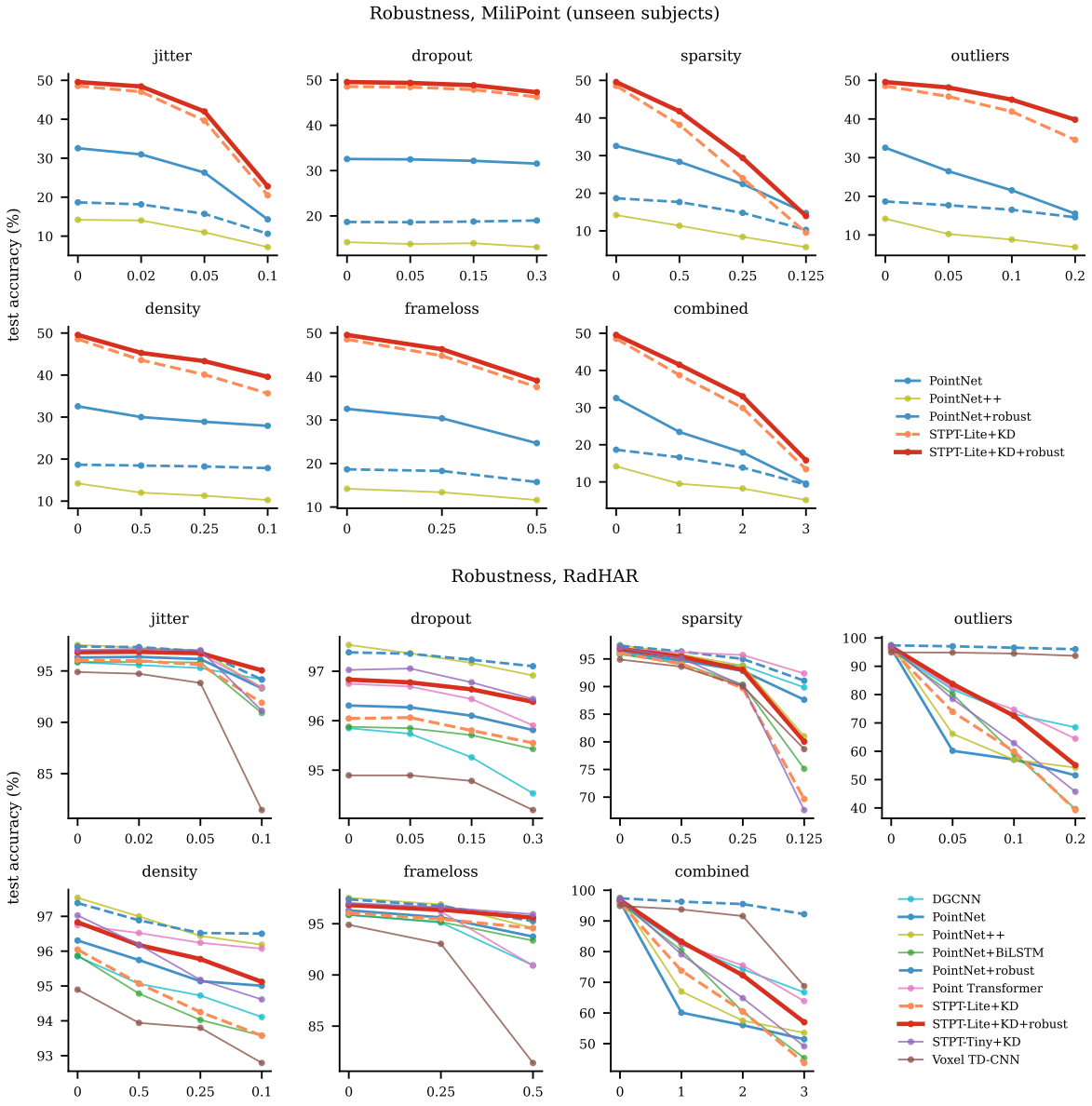


Figure 11.3: Accuracy under controlled perturbations (identical perturbed test samples for every model). Top: MiliPoint; bottom: RadHAR.

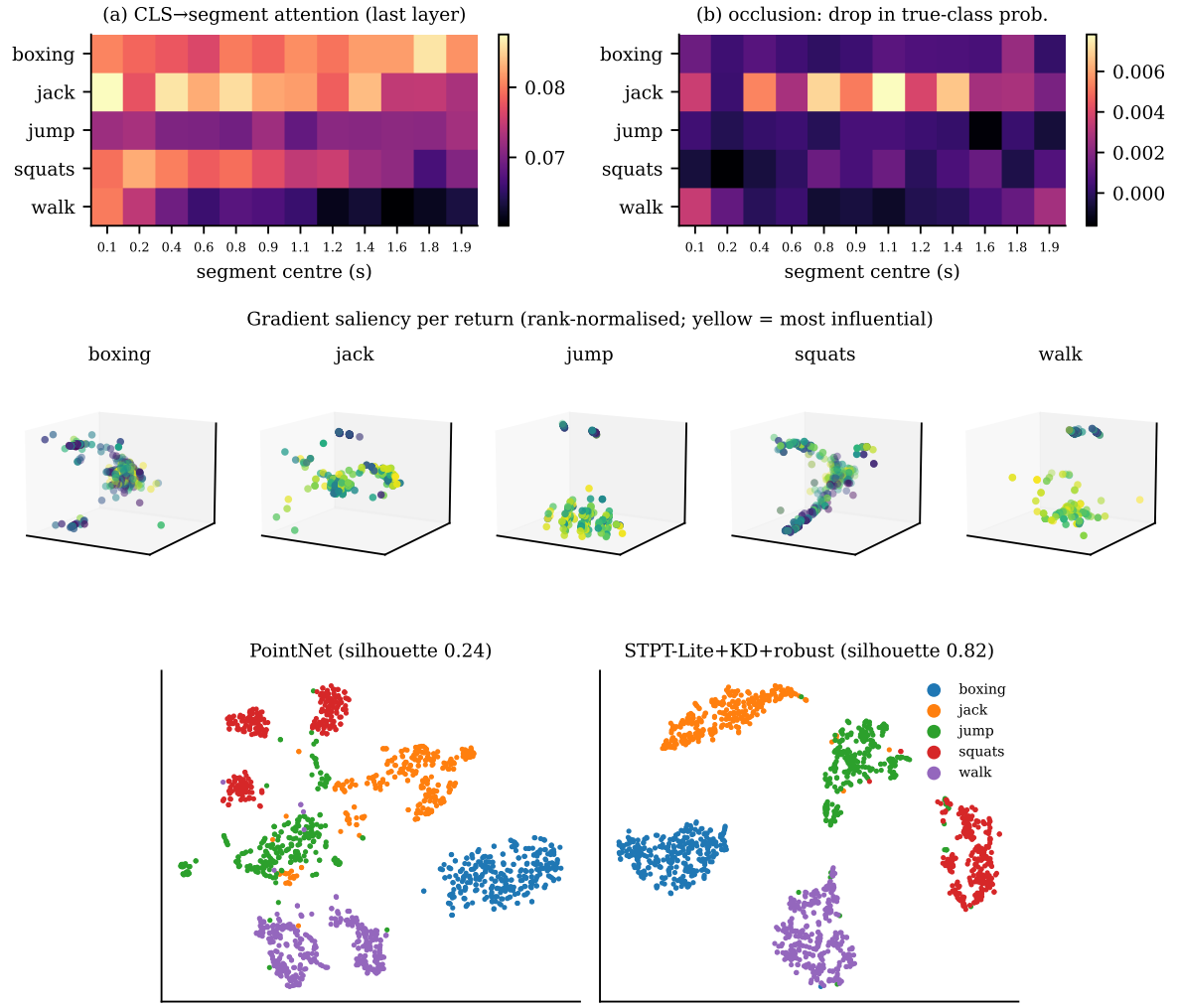


Figure 11.4: Temporal attention vs. occlusion importance, per-point gradient saliency, and t-SNE of embeddings (RadHAR test).

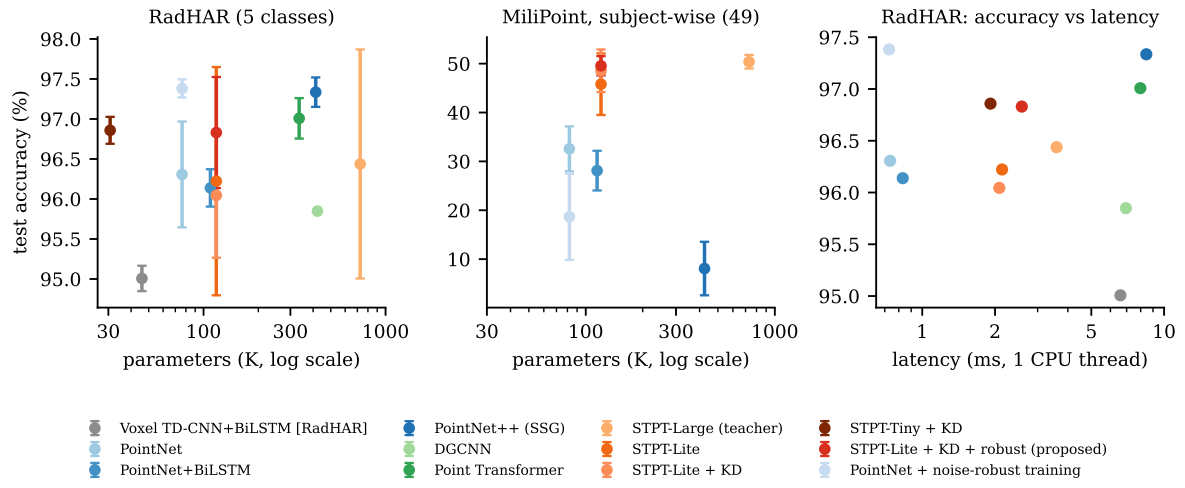


Figure 11.5: Accuracy vs. parameters and latency.

Chapter 12

Working Demonstration

12.1 Purpose and integrity

The demonstration (`demo/app.py`, Streamlit) executes the *actual* exported checkpoint on real radar data in the datasets' native formats. It contains no hard-coded, random or manually assigned predictions: every number on the screen is computed by `inference.Bundle`, which uses the same pre-processing code as training and evaluation.

12.2 How to run

```
pip install -r requirements.txt streamlit plotly
streamlit run demo/app.py          # opens http://localhost:8501
# command-line alternative
python src/inference.py --bundle checkpoints/stpt_lite_radhar \
                        --input demo/samples/radhar_test_jump.txt
```

12.3 Application structure

Sidebar select the trained model (RadHAR or MiliPoint bundle), a bundled test sample (ground truth known) or an uploaded file (`.txt` RadHAR ROS dump, `.npz` MiliPoint frames).

1 Input & pre-processing frame/return/window counts; 3-D view of a raw window with antenna returns, gated clutter and kept person returns marked; the pre-processed model input coloured by time.

2 Recognition recording-level prediction (majority over 2-s windows), ground truth if known, single-window and whole-recording latency, per-window class probabilities and a probability-over-time heat map. A note explains that softmax probabilities are not calibrated accuracy.

3 Noise robustness choose a perturbation (jitter, dropout, sparsity, outliers, variable density, frame loss, combined), its level and seed; the app shows clean vs. noisy point clouds, both predictions, the percentage of unchanged window predictions and, when ground truth is known, the window accuracy before and after. The test-set robustness table is shown below.

4 Model & results parameters, FP32/INT8 size, MFLOPs, latency, held-out test accuracy, macro-F1/precision/recall, confusion matrix, and the full architecture and training configuration.

12.4 Input validation and error handling

Unsupported extensions, text without RadHAR point records, npz files without `pts`/`frame` or with wrong shapes, NaN/inf values, recordings shorter than one 2-s window, and model/data mismatches (e.g. a RadHAR file with the MiliPoint model) are rejected with a message instead of crashing the app.

12.5 Bundled samples

`demo/samples/` contains 12-s excerpts of five RadHAR *official test* recordings (one per class, in the original text format) and six 4-s MiliPoint sequences from *test subjects* 8 and 9. None of them was used for training or model selection.

12.6 Screenshots

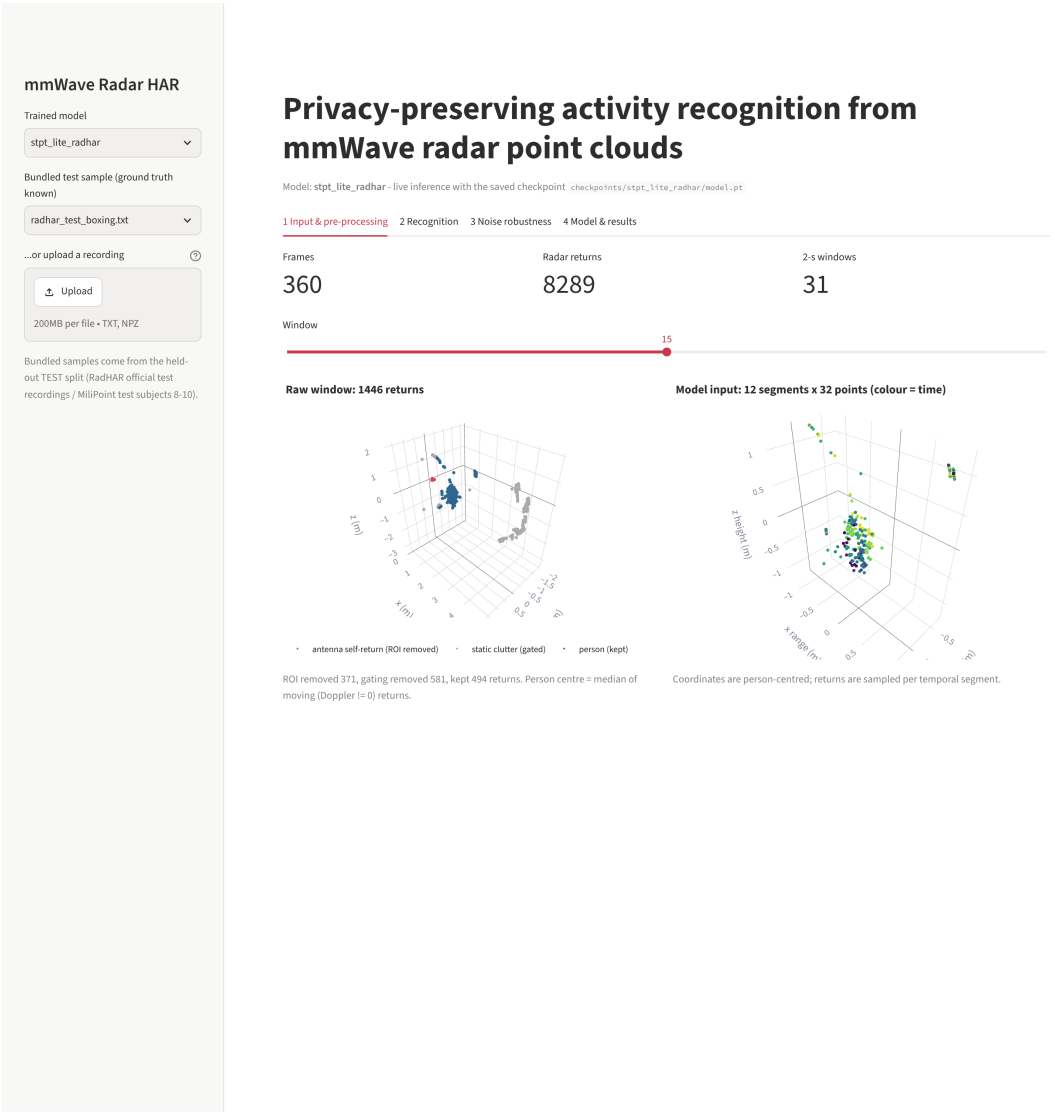


Figure 12.1: Demonstration (real screenshot), Tab 1: raw window with antenna returns, gated clutter and kept person returns; pre-processed model input.

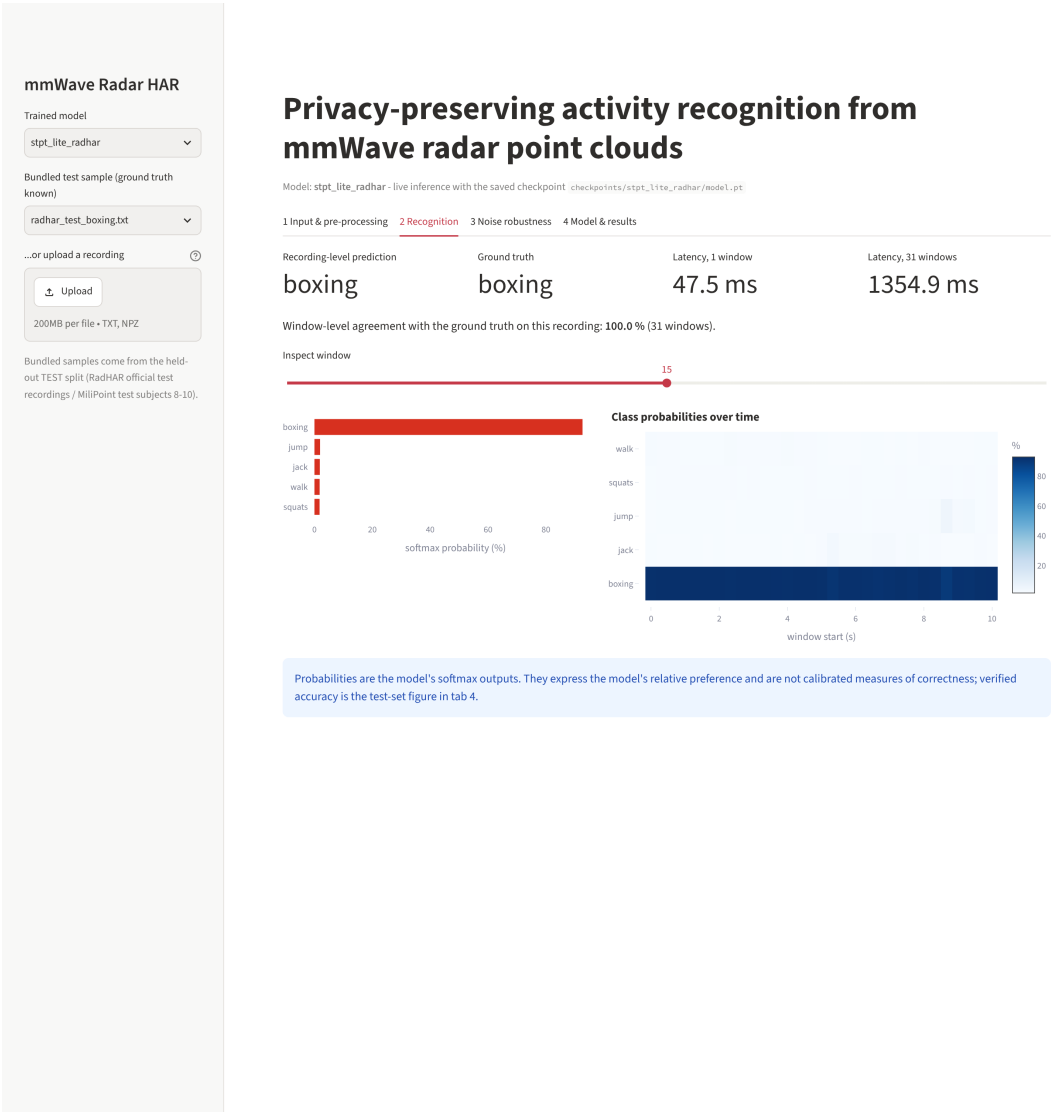


Figure 12.2: Demonstration (real screenshot), Tab 2: live recording-level prediction, latency and per-window class probabilities.

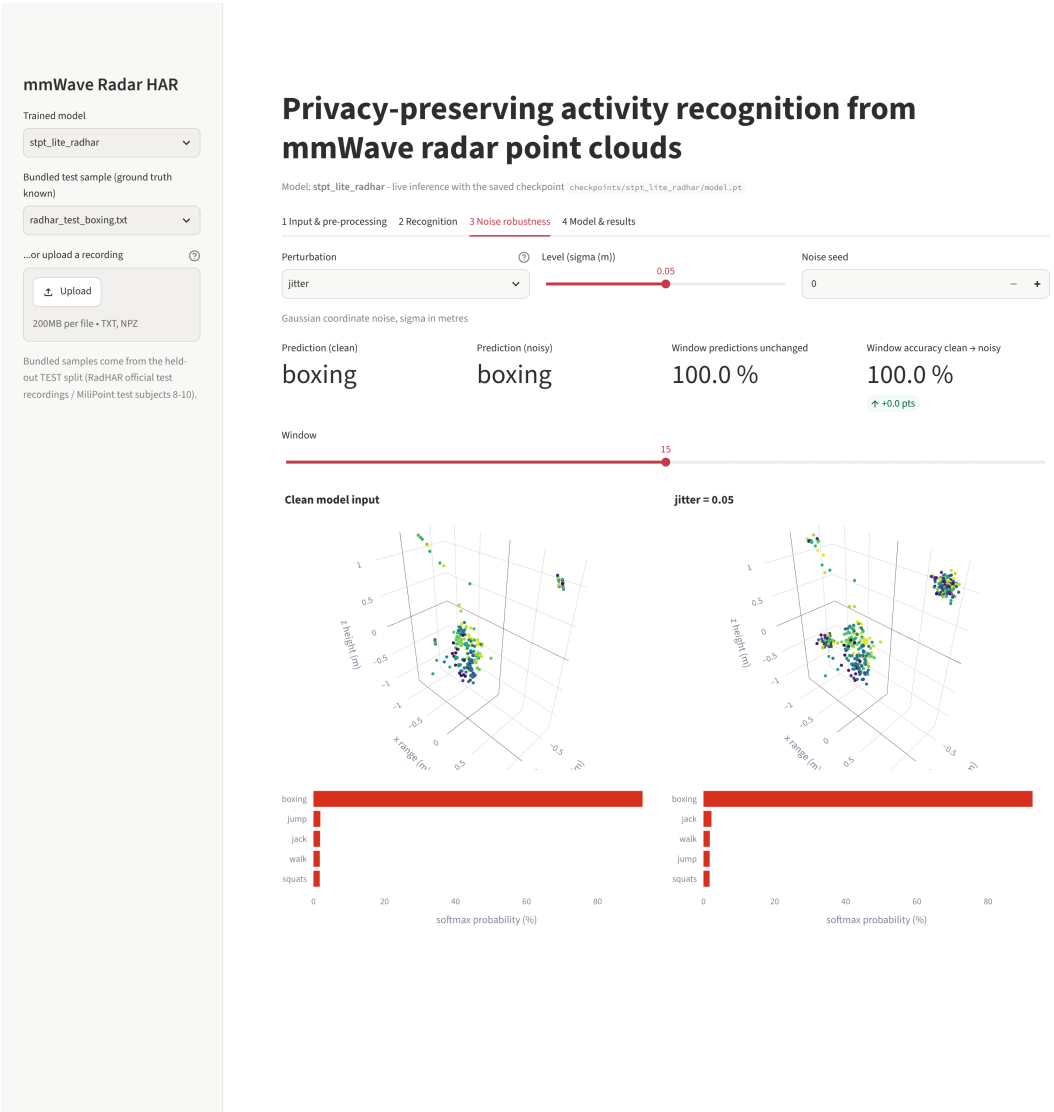


Figure 12.3: Demonstration (real screenshot), Tab 3: controlled noise injection; clean vs. noisy point cloud and predictions.

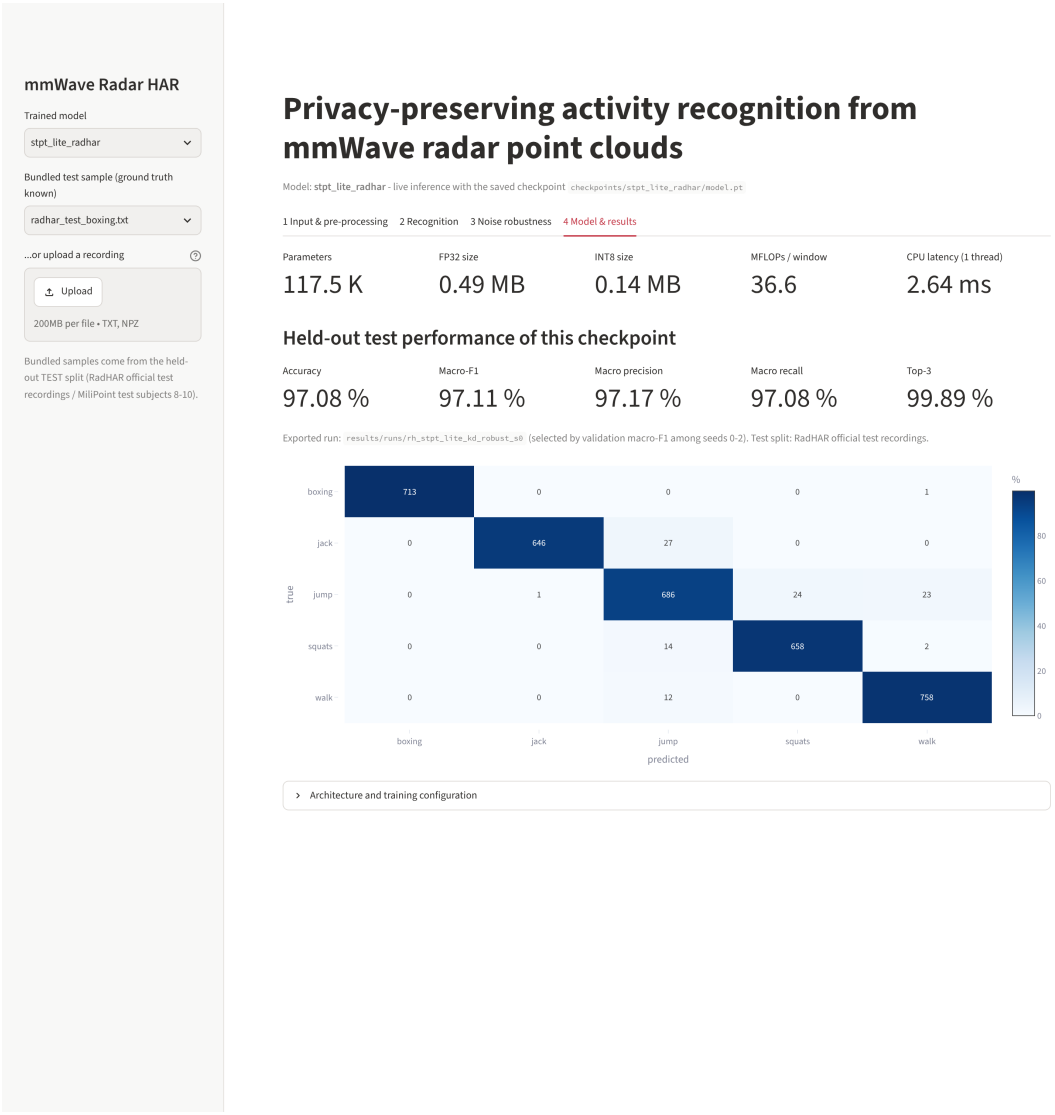


Figure 12.4: Demonstration (real screenshot), Tab 4: model card with held-out test metrics and confusion matrix.

Chapter 13

Testing and Quality Assurance

13.1 Test strategy

Table 13.1: Test levels.

Level	What is tested	How
Unit	parser on a real recording and on garbage; safe unpickler against a malicious pickle; every noise function (shape, determinism, no in-place change, effect size); every model's forward pass with empty segments	`pytest tests/`
Data validation	split disjointness (RadHAR recordings, MiliPoint subjects); tensor shapes and finiteness; deterministic evaluation samples; train-only statistics	`pytest` + leakage audit
Integration	exported bundle → raw recording → probabilities (sum to 1, deterministic); short input rejected	`pytest`, CLI
Checkpoint	bundle reload without retraining; seed selected by validation	CLI + demo
Application	app start, sample loading, preprocessing view, inference, all 7 noise conditions, sample switching, error handling	Streamlit `AppTest`, headless-browser screenshots
Performance	latency (median of 200, batch 1, 1 thread), FLOPs, model size	`train.py`
Reproducibility	identical robustness results on repeated evaluation	robustness determinism check

Table 13.2: Test results (generated by `scripts/make_qa.py`).

Suite	Passed	Failed
Unit / data / integration (`pytest tests/`)	23	0
Demo application (`scripts/apptest_demo.py`, 28 interaction steps)	28	0
Leakage audit checks (`scripts/leakage_audit.py`)	9	0

13.2 Results

The demo test drives the real application headlessly: it loads both exported models, every bundled test sample and every perturbation condition, and fails on any exception or error message shown to the user.

13.3 Quality-assurance matrix

Table 13.3: Final quality-assurance matrix (evidence-based).

Area	Check	Evidence	Status
C1 Literature	$\geq$15 papers 2022–26 (16 of 40 references)	`paper/refs.bib`	**PASS**
C1 Gap	Research gap and contributions stated	`paper Sec. I-II`	**PASS**
C2 Datasets	Both datasets parsed and used	`results/eda_stats.json`	**PASS**
C2 EDA	EDA and pre-processing figures	`figures/eda_*, preproc_*`	**PASS**
C2 Leakage	Automated audit: splits disjoint, train-only statistics	`docs/leakage_audit.md`	**PASS**
C3 Model	Proposed model implemented, diagram	`src/models.py, figures/arch.pdf`	**PASS**
C4 Baselines	$\geq$3 baselines incl. SOTA (6 re-trained)	`results/summary_radhar.csv`	**PASS**
C4 Seeds	Proposed model, 3 seeds per dataset (3/3)	`results/all_runs.csv`	**PASS**
C4 Statistics	McNemar (Holm) and bootstrap CI	`results/stats.json`	**PASS**

Area	Check	Evidence	Status
C4 Cross-dataset	MiliPoint→RadHAR transfer	`results/summary_transfer.csv`	**PASS**
C4 Ablation	Component ablation	`results/summary_ablation.csv`	**PASS**
Robustness	7 perturbation types, 3 severities, both datasets	`results/robustness_*.csv`	**PASS**
Training	Loss/accuracy/LR logged every epoch for every run	`results/runs/*/history.csv`	**PASS**
Metrics	Accuracy, macro P/R/F1, per-class, top-3, latency, size	`metrics.json`	**PASS**
C5 XAI	Attention, occlusion, saliency, t-SNE	`results/xai.json`	**PASS**
Checkpoint	Exported model + inference script	`checkpoints/, src/inference.py`	**PASS**
Tests	pytest (23 passed, 0 failed)	`results/pytest.log`	**PASS**
C7 Demo	Streamlit demo, headless test (28 steps, 0 failed)	`results/apptest.json`	**PASS**
C7 Demo video	Screen recording of the live demo	`figures/demo/demo.mp4`	**PASS**
Config	Hyper-parameters in a config file	`configs/default.yaml`	**PASS**
Documentation	README, DEVLOG, reproduction script	`README.md, docs/`	**PASS**
No placeholders	No pending text in paper or report sources	`placeholder grep`	**PASS**
C6 Paper	Compiled / built and inspected page by page	`paper/main.pdf`	**PASS**
Report	Compiled / built and inspected page by page	`report/report.pdf`	**PASS**
C7 Slides	Compiled / built and inspected page by page	`presentation/Team10_mmWave_HAR.pptx`	**PASS**

Chapter 14

Limitations and Future Enhancements

14.1 Limitations

1. **RadHAR is not a subject- or session-independent benchmark.** It has two subjects, no subject IDs, and test recordings made minutes after training recordings in the same room (Chapter 8). High RadHAR accuracy therefore shows generalisation to new recordings, not to new people or rooms.
2. **MiliPoint**: the radar provides *xyz* only, action names are not published, and the test set has three subjects, so per-subject variance is large and only partly captured.
3. **Synthetic perturbations**: the robustness study perturbs recorded point clouds; it does not simulate the physical radar (multipath geometry, interference, different mounting heights or rooms).
4. **Compute**: all experiments ran on a 2-core CPU. Training was capped at 15 (RadHAR) and 30 (MiliPoint) epochs with early stopping and a 20/24-frame training stride. DGCNN and Point Transformer were evaluated on RadHAR only (DGCNN with one seed, the voxel baseline with two), because of their cost on the larger MiliPoint training set. The voxel baseline uses fewer filters than the original RadHAR network. Ablation and transfer variants use one to three seeds, as stated in each table.
5. **Sensitivity to false detections and extreme sparsity**: on RadHAR, 20% injected outliers reduce the proposed model to 55% (PointNet with the same robust training: 96%), and with 12.5% of the returns kept on MiliPoint it falls to PointNet level (Chapter 11).
6. **Edge claims**: latency was measured on a desktop-class CPU thread and storage is computed for INT8 weights; deployment on a micro-controller was not carried out.
7. **Scope**: no fall detection (no labels), single person only, fixed 2-s windows, no clinical validation.
8. **Explainability**: attention maps are not explanations in a causal sense [30]; occlusion and gradient saliency give complementary but approximate views.

14.2 Future enhancements

1. Collect or adopt a dataset with subject IDs, several rooms and fall events; evaluate leave-one-subject-out and cross-room.
2. Port the model to an embedded runtime (e.g. ONNX Runtime Mobile or an MCU inference library) and measure on the radar's companion processor.
3. Physics-based radar simulation for robustness (multipath, occlusion).

4. Self-supervised pre-training on unlabelled radar data (both datasets contain many unlabelled frames), and continuous (streaming) recognition.

Chapter 15

Conclusion

What was developed. A complete, reproducible pipeline for privacy-preserving activity recognition from mmWave radar point clouds. It comprises safe dataset loaders for RadHAR and MiliPoint, radar-specific pre-processing (antenna-return ROI, Doppler-based person centring, 1.2 m clutter gating), seven model families including the proposed STPT-Lite, a seeded noise-perturbation module, an automated leakage audit, a training/evaluation harness with statistics, an exported inference bundle, a command-line inference tool and an interactive Streamlit demonstration.

Why these methods. Every choice traces back to a measurement (Table 4.2). The clutter removal follows from the range/Doppler analysis. The factorised attention follows from the extreme per-frame sparsity together with strong temporal structure. Recording- and subject-disjoint splits follow from the overlap of windows. The validation cap and the longer MiliPoint schedule follow from observed failures (Chapter 10).

What was demonstrated. On 49 MiliPoint actions performed by unseen people, the proposed STPT-Lite+KD+robust reaches 49.56% top-1 (distillation alone: 48.55%), about 17 points above the best re-implemented baseline, with 117.5 K parameters, 36.6 MFLOPs, 2.58 ms CPU latency and 0.14 MB of INT8 weights. Noise-robust training improves accuracy under every perturbation type on MiliPoint without lowering clean accuracy. On RadHAR all models saturate at about 95–98%. Pre-training on MiliPoint raises RadHAR accuracy with 10% of the training recordings from 61.22% to 78.98%. On unseen people, removing the intra-segment attention or the temporal Transformer costs several points.

What was verified. Split disjointness, absence of duplicated recordings and frames, train-only statistics and validation-only checkpoint selection (automated audit). The tests pass, the saved model reloads and runs on raw files, and the demo works end to end (application test harness and real-browser screenshots).

What remains unresolved. RadHAR cannot test new users or rooms. Our PointNet++ re-implementation under-performs on MiliPoint. Under extreme sparsity STPT-Lite falls to PointNet level, and on RadHAR it is much more sensitive to injected false detections than PointNet with robust training. No on-device deployment or clinical evaluation was performed (Chapter 14).

Bibliography

[1] H. Cui, S. Zhong, J. Wu, Z. Shen, N. Dahnoun, and Y. Zhao, "MiliPoint: A point cloud dataset for mmwave radar," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[2] V. G. Zakka, L. J. Manso, and Z. Dai, "Multi-head adaptive graph convolution network for sparse point cloud-based human activity recognition," arXiv:2504.02778, 2025.

[3] I. Ullmann, R. G. Guendel, N. C. Kruse, F. Fioranelli, and A. Yarovoy, "A survey on radar-based continuous human activity recognition," *IEEE J. Microwaves*, vol. 3, no. 3, pp. 938–950, 2023.

[4] A. D. Singh, S. S. Sandha, L. Garcia, and M. Srivastava, "RadHAR: Human activity recognition from point clouds generated through a millimeter-wave radar," in *Proc. 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems (mmNets)*, 2019, pp. 51–56.

[5] P. Gong, C. Wang, and L. Zhang, "MMPoint-GNN: Graph neural network with dynamic edges for human activity recognition through a millimeter-wave radar," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2021.

[6] G. Lee and J. Kim, "Improving human activity recognition for sparse radar point clouds: A graph neural network model with pre-trained 3D human-joint coordinates," *Applied Sciences*, vol. 12, no. 4, p. 2168, 2022.

[7] D. Salami, R. Hasibi, S. Palipana, P. Popovski, T. Michoel, and S. Sigg, "Tesla-Rapture: A lightweight gesture recognition system from mmwave radar sparse point clouds," *IEEE Trans. Mobile Computing*, vol. 22, no. 8, pp. 4946–4960, 2023.

[8] S. Palipana, D. Salami, L. A. Leiva, and S. Sigg, "Pantomime: Mid-air gesture recognition with sparse millimeter-wave radar point clouds," *Proc. ACM Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 5, no. 1, 2021.

[9] Z. Gu, X. He, G. Fang, C. Xu, F. Xia, and W. Jia, "Millimeter wave radar-based human activity recognition for healthcare monitoring robot," arXiv:2405.01882, 2024.

[10] S. Lim, C. Park, S. Lee, and Y. Jung, "Human activity recognition based on point clouds from millimeter-wave radar," *Applied Sciences*, vol. 14, no. 22, p. 10764, 2024.

[11] J. Yan, Y. Wang, C. Xu, and D. Liu, "OG-PCL: Efficient sparse point cloud processing for human activity recognition," arXiv:2511.08910, 2025.

[12] S. Gao, J. Zhang, L. Mei, S. Wang, and X. Wang, "Exploring spatial-temporal representation via star graph for mmwave radar-based human activity recognition," arXiv:2512.12013, 2025.

[13] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660.

[14] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[15] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graphics*, vol. 38, no. 5, pp. 146:1–146:12, 2019.

[16] H. Zhao, L. Jiang, J. Jia, P. Torr, and V. Koltun, "Point transformer," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 16 259–16 268.

[17] M.-H. Guo, J.-X. Cai, Z.-N. Liu, T.-J. Mu, R. R. Martin, and S.-M. Hu, "PCT: Point cloud transformer," *Computational Visual Media*, vol. 7, no. 2, pp. 187–199, 2021.

[18] X. Wu, Y. Lao, L. Jiang, X. Liu, and H. Zhao, "Point transformer V2: Grouped vector attention and partition-based pooling," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[19] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer V3: Simpler, faster, stronger," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[20] G. Qian, Y. Li, H. Peng, J. Mai, H. Hammoud, M. Elhoseiny, and B. Ghanem, "PointNeXt: Revisiting PointNet++ with improved training and scaling strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[21] X. Ma, C. Qin, H. You, H. Ran, and Y. Fu, "Rethinking network design and local geometry in point cloud: A simple residual MLP framework," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[22] H. Fan, Y. Yang, and M. Kankanhalli, "Point 4D transformer networks for spatio-temporal modeling in point cloud videos," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 204–14 213.

[23] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[24] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[25] J. Yang, H. Huang, Y. Zhou, X. Chen, Y. Xu, S. Yuan, H. Zou, C. X. Lu, and L. Xie, "MM-Fi: Multi-modal non-intrusive 4D human dataset for versatile wireless sensing," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[26] A. Chen, X. Wang, S. Zhu, Y. Li, J. Chen, and Q. Ye, "mmBody benchmark: 3D body reconstruction dataset and analysis for millimeter wave radar," in *Proc. 30th ACM Int. Conf. Multimedia (MM)*, 2022.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[29] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[30] S. Jain and B. C. Wallace, "Attention is not explanation," in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 3543–3556.

Appendix A

Configuration, Installation and Commands

A.1 Default configuration (`configs/default.yaml`)

```
# Default hyper-parameters for src/train.py (command-line flags override
    these).
# Chosen once, before any test-set evaluation; identical for every model
    .
optimiser: {lr: 0.002, wd: 0.05, bs: 64, schedule: one-cycle (10 % warm-
    up)}
epochs: 15            # upper bound; early stopping on validation macro-
    F1
patience: 5
label_smoothing: 0.1
grad_clip: 1.0
G: 12                 # temporal segments per 2-s window
P: 32                 # points sampled per segment
gate: 1.2             # m, horizontal gating radius around the person (
    RadHAR)
feats: full           # full = x,y,z,Doppler,intensity,time ; xyz =
    Doppler/intensity zeroed
aug: std              # std = rotation+scale+1 cm jitter ; nojitter ;
    none
robust: 0             # 1 = add noise-robust augmentation (src/noise.py:
    robust_augment)
kd_alpha: 0.7
kd_T: 4.0
```

A.2 Installation

```
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\
    Scripts\activate
pip install -r requirements.txt streamlit plotly pytest
```

A.3 Data preparation

```
git clone --depth 1 https://github.com/nesl/RadHAR ext/RadHAR
git clone --depth 1 https://github.com/yizzfz/MiliPoint ext/MiliPoint
```

```
# download MiliPoint_data.zip (link in the MiliPoint README) and unzip
    to ext/milipoint_raw/
python src/radhar_parse.py ext/RadHAR/Data data/processed/radhar_frames.
    npy
python scripts/eda.py
```

A.4 Training and evaluation

```
# one run
python src/train.py --data radhar --model stpt_lite --seed 0 \
       --teacher results/runs/rh_stpt_teacher_s0 --robust 1 --tag
    rh_stpt_lite_kd_robust_s0
# all runs of the study (2 parallel workers)
python scripts/runner.py scripts/jobs_main.txt 2
# tables, statistics, robustness, explainability, audit
python scripts/analyze.py
python scripts/xai_robust.py robust radhar ; python scripts/xai_robust.
    py robust milipoint
python scripts/xai_robust.py robustfig ; python scripts/xai_robust.py
    xai
python scripts/leakage_audit.py ; python scripts/make_numbers.py
python -m pytest -q tests
```

A.5 Inference and demonstration

```
python scripts/export_model.py rh_stpt_lite_kd_robust checkpoints/
    stpt_lite_radhar radhar
python src/inference.py --bundle checkpoints/stpt_lite_radhar \
       --input demo/samples/radhar_test_walk.txt
streamlit run demo/app.py
```

Appendix B

Test Log

```
.......................                                                  [100%]
================================== PASSES ==================================
=========================== short test summary info ============================
PASSED tests/test_pipeline.py::test_parse_sample_recording
PASSED tests/test_pipeline.py::test_parse_rejects_garbage
PASSED tests/test_pipeline.py::test_safe_pickle_blocks_code
PASSED tests/test_pipeline.py::test_radhar_splits_disjoint_and_shapes
PASSED tests/test_pipeline.py::test_milipoint_subject_disjoint
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[jitter]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[dropout]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[sparsity]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[outliers]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[density]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[frameloss]
PASSED tests/test_pipeline.py::test_noise_shapes_and_determinism[combined]
PASSED tests/test_pipeline.py::test_noise_effects
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[pointnet]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[pointnet_lstm]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[pointnet2]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[dgcnn]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[pointtransformer]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[voxel_tdcnn]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[stpt_lite]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[stpt_tiny]
PASSED tests/test_pipeline.py::test_model_forward_with_empty_segments[stpt_teacher]
PASSED tests/test_pipeline.py::test_bundle_inference_on_real_sample
23 passed in 1.38s
```